



UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELLE TECNOLOGIE
DELL'INFORMAZIONE
CORSO DI LAUREA IN INFORMATICA

Ingegneria del Software

Anno Accademico 2025–2026

Elaborato di gruppo

BugBoard26

Candidati

Molinari Rocco - N86005071

Megna Daniele - N86005120

Docenti

Prof. Porfirio Tramontana

Prof. Bernardo Breve

Napoli, 21 Luglio 2026

Indice

1	Documento di Specifica dei Requisiti Software	4
1.1	Introduzione	4
1.1.1	Scopo	4
1.1.2	Descrizione del sistema	4
1.2	Glossario	4
1.3	Target utenti	5
1.3.1	Admin	6
1.3.2	User	6
1.3.3	Utente Readonly	6
1.4	Use Case Diagram	6
1.5	Requisiti funzionali	8
1.5.1	RF-01 Gestione Utenze e Autenticazione	8
1.5.2	RF-02 Creazione e Gestione Issue	8
1.5.3	RF-03 Visualizzazione e Filtraggio	9
1.5.4	RF-04 Assegnazione e Notifiche	9
1.5.5	RF-05 Gestione Etichette	9
1.5.6	RF-06 Ruolo Readonly	9
1.5.7	RF-07 Scadenze	9
1.6	Requisiti non funzionali	9
1.6.1	RNF-01 Sicurezza e Riservatezza	9
1.6.2	RNF-02 Integrità dei Dati e Gestione dei Conflitti	10
1.6.3	RNF-03 Affidabilità e Continuità del Servizio	10
1.6.4	RNF-04 Prestazioni, Scalabilità e Capacità	11
1.6.5	RNF-05 Manutenibilità ed Estensibilità	12
1.6.6	RNF-06 Portabilità e Compatibilità	12
1.7	Requisiti di sistema e vincoli progettuali	12
1.7.1	RS-01 Architettura Distribuita	12
1.7.2	RS-02 Architettura API REST e Disaccoppiamento	13
1.7.3	RS-03 Tecnologie Adottate	13
1.7.4	RS-04 Gestione dello Stato e Persistenza	13
1.7.5	RS-05 Separazione delle Responsabilità	14

1.7.6	RS-06 Meccanismo di Versioning delle Issue	14
1.7.7	RS-07 Ambiente di Deployment e Containerizzazione	14
1.7.8	RS-08 Prevenzione Cross-Site Scripting (XSS)	14
1.7.9	RS-09 Prevenzione SQL Injection	15
1.8	Casi d'uso dettagliati (Modello Cockburn) e relativi mock-up	15
1.8.1	UC-01 Creazione Issue	15
1.8.2	UC-02 Assegnazione Issue	17
2	Documento di Design del Sistema	20
2.1	Descrizione dell'Architettura Proposta e Criteri di Design	20
2.1.1	Criteri di Design Adottati e Motivazioni	20
2.2	Descrizione e Motivazione delle Scelte Tecnologiche	21
2.2.1	Infrastruttura Cloud: Google Cloud Platform (GCP)	21
2.2.2	Ambiente di Back-end: Java, Spring Boot e PostgreSQL	21
2.2.3	Ambiente di Front-end: Vue.js 3, Vite, Axios e Tailwind CSS	22
2.3	Diagramma di Deployment	23
2.3.1	Razionale della Distribuzione su Tre Nodi	23
2.4	Strategia di DevOps: Containerizzazione (Docker) e CI/CD (GitHub Actions)	24
3	Design del Software	25
3.1	Architettura Generale a Livelli (Layered Architecture)	25
3.1.1	Gestione Centralizzata degli Errori	26
3.1.2	Flusso di Autenticazione tra i Livelli	27
3.1.3	Ottimizzazioni del Livello di Accesso ai Dati	27
3.2	Descrizione dello Schema per la Persistenza Dati	27
3.2.1	Entità Utente	28
3.2.2	Entità Issue	28
3.2.3	Entità Sessione e Notifica	29
3.3	Diagramma delle Classi di Design	29
3.4	Descrizione e Motivazione delle Scelte di Software Design	35
3.4.1	Pattern Builder	35
3.4.2	Pattern Observer e Disaccoppiamento Event-Driven	36
3.4.3	Pattern Data Transfer Object (DTO) e Mappers	36
3.4.4	Dependency Inversion Principle (DIP) e Inversione del Controllo (IoC)	36
3.4.5	Interface Segregation e Separazione Comando/Interrogazione	37
3.5	Evidenza dell'Uso di Strumenti di Versioning	37
3.5.1	Analisi dei Commit e Frequenza Temporale	37
3.5.2	Contributori e Ripartizione del Lavoro	38
3.6	Report di Qualità del Codice	38
3.6.1	Analisi dei Risultati e Debito Tecnico	39

3.6.2	Dettaglio dell'Analisi Complessiva (Overall Code)	40
3.7	Codice Sorgente Sviluppato e Struttura di Build	40
3.7.1	Descrittore di Build Back-end: pom.xml	41
3.7.2	Containerizzazione del Back-end: backend/Dockerfile	42
3.7.3	Descrittore di Build Front-end: package.json	44
3.7.4	Containerizzazione del Front-end: frontend/Dockerfile	44
3.7.5	Orchestrazione Locale Multi-Container: docker-compose.yml	45
4	Attività di Testing	47
4.1	Strumenti e ambiente di test	47
4.1.1	Livello e approccio di test	47
4.2	Test Plan	48
4.2.1	Ambiente di test (comune)	48
4.2.2	Assegnazione di una issue (assignIssueToUser)	48
4.2.3	Aggiunta di un commento (aggiungiCommento)	50
4.2.4	Eliminazione di un utente (deleteUser)	51
4.2.5	Criteri di superamento (comuni)	52
4.3	Codice dei Test di Unità Automatici	53
4.3.1	Estratto di IssueServiceTest.java	53
4.3.2	Estratto di UserServiceTest.java	67
4.4	Strategie di progettazione dei test	71
4.4.1	Metodi sotto test	71
4.4.2	Testing black-box: classi di equivalenza (criterio each-choice)	71
4.4.3	Testing white-box: criteri di copertura strutturale	73

1 Documento di Specifica dei Requisiti Software

Il presente capitolo definisce in modo formale ed esaustivo i requisiti software del sistema BugBoard26. L'obiettivo consiste nel delineare accuratamente le funzionalità, i vincoli operativi e le qualità complessive che il sistema deve garantire, fornendo una linea guida univoca per le successive fasi di progettazione, implementazione e verifica del software.

1.1 Introduzione

1.1.1 Scopo

Il presente documento descrive la specifica dei requisiti software del sistema BugBoard26, sviluppato per il corso di Ingegneria del Software. All'interno del documento troviamo funzionalità del sistema, glossario, target degli utenti, casi d'uso, requisiti funzionali, non funzionali e di sistema, inoltre troviamo anche la formalizzazione di due casi d'uso tramite descrizione testuale strutturata e prototipazione visuale dell'interfaccia utente.

1.1.2 Descrizione del sistema

Il sistema si occupa della gestione delle issue all'interno di un'organizzazione. Inoltre il sistema permette di:

- segnalare bug, feature, question, documentation;
- assegnare le issue agli utenti;
- controllare stato e sviluppo di ogni issue;
- aggiungere commenti alle relative issue.

1.2 Glossario

Sono riportati i termini tecnici presenti nel documento e nel sistema BugBoard26.

Termine	Descrizione
---------	-------------

Issue	Segnalazione relativa ad un problema o richiesta software
Bug	Issue relativa ad un malfunzionamento
Admin	Utente con privilegi amministrativi, può gestire gli utenti, accedere a tutte le issue del sistema e assegnarle
User	Utente normale del sistema, può interagire con le sole issue assegnate all'interno del sistema, eventualmente generandole
Creatore	Utente che crea la issue; nello schema corrisponde al campo idcreatore
Assegnatario	Amministratore che effettua l'assegnazione e prende in carico la issue; nello schema corrisponde al campo idassegnatario
Destinatario	Utente con ruolo User a cui la issue è assegnata per la risoluzione; nello schema corrisponde al campo assegnato_a
Readonly	Utente con permessi di sola visualizzazione di issue di tipo bug e lettura delle relative informazioni
TODO	Stato di una issue appena creata
IN_PROGRESS	Stato di una issue appena assegnata
DONE	Stato di una issue completata da uno user
Expired	Stato di una issue quando è passata la data di scadenza; l'admin può decidere di riassegnarla
Closed	Stato di una issue chiusa da un admin e che non deve essere più modificata e riassegnata
Notifica	Avviso generato dopo l'assegnazione di una issue da parte di un amministratore ad un utente
Versioning	Meccanismo per evitare modifiche concorrenti sulla stessa issue

1.3 Target utenti

Gli utenti del sistema hanno diversi compiti in base al ruolo, i ruoli sono:

- admin
- user
- readonly

1.3.1 Admin

L'admin può:

- creare utenti;
- assegnare issue;
- impostare scadenze;
- visualizzare tutte le issue;
- gestire lo stato delle issue;
- eliminare utenti.

1.3.2 User

L'user può:

- creare issue;
- visualizzare issue assegnate;
- aggiungere commenti;
- impostare issue in stato done (completata).

1.3.3 Utente Readonly

L'utente readonly non ha facoltà di modificare informazioni, ma può:

- visualizzare i bug con le relative informazioni;
- leggere i commenti associati ai bug.

1.4 Use Case Diagram

Il diagramma dei casi d'uso mostrato in Figura 1.1 descrive le macro-funzionalità del sistema BugBoard26 e le interazioni tra gli attori Admin, User e Readonly e il sistema stesso.



Figura 1.1: Diagramma dei casi d'uso del sistema BugBoard26

I principali casi d'uso mostrati nel diagramma sono:

- Autenticazione utente
- Gestione utenti
- Creazione issue
- Visualizzazione issue Globale
- Visualizzazione issue Utente
- Visualizzazione issue tipo bug e commenti
- Filtraggio issue Admin
- Filtraggio issue User
- Filtraggio bug readonly
- Assegnazione issue
- Notifica di assegnazione
- Inserimento commenti

- Gestione etichette
- Impostazione data scadenza

1.5 Requisiti funzionali

I requisiti funzionali descrivono le funzionalità offerte dal sistema BugBoard26.

1.5.1 RF-01 Gestione Utente e Autenticazione

RF-01.1 Autenticazione: Il sistema deve permettere agli utenti di autenticarsi inserendo email e password.

RF-01.2 Account di Default: Il sistema deve essere fornito con un account amministratore (Admin) preconfigurato con credenziali di default.

RF-01.3 Creazione Utenti: Un utente con ruolo Admin deve poter creare nuove utenze nel sistema specificando: nome, cognome, email, password e ruolo (Admin, User, Readonly).

1.5.2 RF-02 Creazione e Gestione Issue

RF-02.1 Creazione Issue: Ogni utente autenticato con ruolo User deve poter segnalare una issue inserendo obbligatoriamente:

- Titolo;
- Descrizione;
- Tipologia (scelta tra: bug, question, documentation, feature).

RF-02.2 Attributi Opzionali Issue: Durante la creazione o modifica di una issue, l'utente deve poter inserire i seguenti dati opzionali:

- Priorità;
- Etichette;
- Allegato (immagine).

RF-02.3 Stato Iniziale: Ogni nuova issue creata deve essere impostata automaticamente dal sistema nello stato TODO.

1.5.3 RF-03 Visualizzazione e Filtraggio

RF-03.1 Vista Riepilogativa: Il sistema deve mostrare una vista elenco di tutte le issue presenti (per Amministratore), delle sole issue assegnate (per User), di tutte le issue di tipo bug (Readonly).

RF-03.2 Filtraggio e Ordinamento: Il sistema deve permettere di filtrare e ordinare la vista riepilogativa in base a: tipologia, stato e priorità.

1.5.4 RF-04 Assegnazione e Notifiche

RF-04.1 Assegnazione Issue: Un utente Admin deve poter assegnare una issue a un utente con ruolo User.

RF-04.2 Notifica Assegnazione: All'atto dell'assegnazione di una issue, il sistema deve generare e inviare una notifica all'utente destinatario dell'assegnazione.

RF-04.3 Visualizzazione Issue Assegnate: Ogni User deve poter visualizzare la lista delle issue a lui assegnate.

1.5.5 RF-05 Gestione Etichette

RF-05.1 Associazione Etichette: Il sistema deve permettere di associare un numero variabile di etichette di testo personalizzabili (es. "frontend", "urgente") a una issue.

1.5.6 RF-06 Ruolo Readonly

RF-06.1 Permessi Sola Lettura: Il sistema deve prevedere il ruolo Readonly. Gli utenti con questo ruolo possono solo visualizzare le issue di tipo bug e i relativi commenti.

1.5.7 RF-07 Scadenze

RF-07.1 Impostazione Scadenze: Un utente Admin deve poter associare una data di scadenza opzionale per la risoluzione di una specifica issue, con possibilità di riassegnazione in caso di mancata consegna entro i limiti di tempo.

1.6 Requisiti non funzionali

I requisiti non funzionali definiscono le qualità del sistema, i vincoli operativi e i criteri di affidabilità che il sistema BugBoard26 deve garantire.

1.6.1 RNF-01 Sicurezza e Riservatezza

Il sistema deve garantire la riservatezza delle credenziali e accesso protetto alle risorse.

- Le password degli utenti non devono mai essere memorizzate in chiaro nel database.
- Accesso alle funzionalità operative (es. creazione e assegnazione issue) deve essere vincolato alla corretta autorizzazione dei ruoli degli utenti autenticati.

1.6.2 RNF-02 Integrità dei Dati e Gestione dei Conflitti

Il sistema deve garantire la consistenza dello stato dell'applicazione, impedendo la corruzione delle informazioni a fronte di accessi simultanei o fallimenti di rete:

- Il sistema deve gestire le modifiche concorrenti sulle medesime issue per evitare il problema dell'aggiornamento perduto.
- In caso di conflitto indotto da un accesso simultaneo alla stessa issue, il sistema deve bloccare l'operazione intercettando l'anomalia e informare l'utente attraverso un messaggio di errore esplicito.

1.6.3 RNF-03 Affidabilità e Continuità del Servizio

Il sistema deve essere robusto e tollerante ai guasti. I requisiti di affidabilità sono espressi tramite soglie quantitative misurabili:

- **Disponibilità (Availability):** tasso di disponibilità mensile non inferiore al 99.5% (escluse le finestre di manutenzione programmata), equivalente al rapporto $MTBF/(MTBF + MTTR)$ tra tempo medio tra i guasti e tempo medio di ripristino.
- **Tempo di Ripristino RTO (Recovery Time Objective):** a fronte del crash di un nodo applicativo, il servizio deve tornare operativo entro 15 minuti. L'obiettivo è reso praticabile dall'immutabilità delle immagini Docker: un nodo guasto viene ricreato dall'immagine pubblicata su GHCR, senza ricostruzione manuale dello stato.
- **Perdita Massima di Dati RPO (Recovery Point Objective):** in caso di guasto del nodo di persistenza, la finestra di dati eventualmente perduti non deve eccedere le 24 ore, da garantire mediante backup periodici del Database.
- **Auto-ripristino (Self-healing):** i container applicativi devono ripartire automaticamente in caso di terminazione anomala del processo, senza intervento manuale (i servizi di produzione adottano la policy restart: unless-stopped).
- **Integrità Transazionale (Rollback):** in caso di interruzione anomala di una procedura di scrittura, il sistema deve garantire il ripristino automatico dello stato precedente (Rollback), impedendo la persistenza di dati parziali o inconsistenti.
- **Tasso di Errore (Error Budget):** la quota di richieste fallite per errori lato server (HTTP 5xx) non deve superare lo 0.5% del totale su base mensile.

1.6.4 RNF-04 Prestazioni, Scalabilità e Capacità

Il sistema deve essere dimensionato per gestire carichi di lavoro concorrenti mantenendo tempi di risposta prevedibili. I requisiti sono espressi tramite percentili e soglie misurabili, anziché tramite un singolo valore massimo:

- **Capacità:** il sistema deve supportare almeno 100 utenti registrati, garantendo la stabilità operativa con un picco di 20 utenti attivi contemporaneamente.
- **Tempi di risposta (latenza):** per le operazioni standard di lettura e scrittura, sotto il carico nominale di 20 utenti concorrenti e in normali condizioni di rete, il percentile 95 (p95) della latenza di risposta deve restare entro 1 secondo, soglia oltre la quale, secondo i limiti di percezione di Nielsen, l'utente avverte un'interruzione del proprio flusso di interazione e il percentile 99 (p99) entro 2 secondi. Le operazioni di upload di un'immagine, più onerose, ammettono una soglia p95 di 3 secondi.
- **Throughput:** il sistema deve sostenere il volume di richieste generato dai 20 utenti attivi concorrenti senza degrado della latenza oltre le soglie indicate.
- **Limiti di payload:** la dimensione di un allegato non deve eccedere 10 MB per file e 12 MB per richiesta, vincolo applicato sia dal Back-end sia dal reverse proxy, così da limitare il costo massimo di una singola richiesta e prevenire la saturazione delle risorse.
- **Utilizzo delle risorse:** l'occupazione di memoria heap del processo Back-end deve restare entro il 75% della RAM allocata al container, mantenendo un consumo stabile (assenza di memory leak) sotto carico prolungato.
- **Efficienza dell'aggiornamento periodico:** l'aggiornamento automatico della dashboard non deve ricaricare l'intero dataset a ogni ciclo, ma trasferire i soli delta (cfr. il meccanismo basato su version), mantenendo pressoché costante il costo del polling al crescere del numero di issue.

Verifica sperimentale. Le soglie di latenza e throughput sono state validate con un test di carico eseguito da un client esterno attraverso il reverse proxy (300 richieste per endpoint in modalità sequenziale e 20 client concorrenti per 15 secondi). Le operazioni di lettura, lista delle issue, polling delle versioni e lista utente hanno fatto registrare una latenza di risposta end-to-end con p95 $\approx$ 160 ms e p99 entro 165 ms a singolo client; tali valori si sono mantenuti sostanzialmente invariati sotto il carico di 20 client concorrenti (p95 $\approx$ 165 ms, p99 $\approx$ 300 ms), con un throughput sostenuto di circa 126 richieste al secondo. Le misure rientrano ampiamente entro le soglie fissate. La latenza osservata è dominata dalla componente di rete e dall'inoltro del reverse proxy, mentre l'elaborazione lato server risulta marginale al volume di dati attuale; l'autenticazione costituisce un caso a parte ($\approx$ 660 ms) per via dell'hashing BCrypt, oneroso per scelta di sicurezza.

1.6.5 RNF-05 Manutenibilità ed Estensibilità

Il sistema deve essere progettato per garantire alti livelli di manutenibilità, facilitando futuri interventi di correzione dei bug o di estensione delle funzionalità.

- **Separazione delle responsabilità:** Il codice sorgente deve isolare la logica di business dalle interfacce di comunicazione e persistenza, minimizzando l'accoppiamento.
- **Autodocumentazione e Standard:** Il codice deve seguire le convenzioni di naming standard del linguaggio utilizzato (Java Code Conventions) e le API REST devono essere documentate in modo chiaro, rendendo il sistema facilmente comprensibile da sviluppatori terzi.

1.6.6 RNF-06 Portabilità e Compatibilità

L'interfaccia utente web (Front-end) deve essere fruibile correttamente sui principali browser web moderni (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) senza richiedere l'installazione di plug-in di terze parti.

1.7 Requisiti di sistema e vincoli progettuali

I requisiti di sistema descrivono i vincoli architetturali, tecnologici e progettuali che devono essere tassativamente rispettati per la realizzazione del sistema BugBoard26.

1.7.1 RS-01 Architettura Distribuita

Il sistema deve essere realizzato secondo un'architettura distribuita client-server. Il sistema deve prevedere la separazione di tre macro-componenti indipendenti:

- **Front-end:** Responsabile esclusivamente dell'interfaccia utente (livello di presentazione) e dell'interazione con l'utente.
- **Back-end:** Responsabile della logica applicativa, della gestione dello stato del sistema.
- **Database:** Responsabile della persistenza dei dati e della integrità referenziale delle entità del sistema.

Le tre componenti devono poter essere sviluppate, eseguite, modificate e distribuite in maniera completamente indipendente.

1.7.2 RS-02 Architettura API REST e Disaccoppiamento

La comunicazione tra il componente Front-end e il componente Back-end deve avvenire esclusivamente tramite interfacce di rete attraverso lo standard API REST esposto dal Back-end. Il Front-end non deve avere alcun accesso diretto al database o al file system del server.

1.7.3 RS-03 Tecnologie Adottate

Il sistema deve essere sviluppato utilizzando tecnologie orientate agli oggetti e framework per lo sviluppo web moderni. Nello specifico, lo stack tecnologico è vincolato a:

- **Back-end:**

- Linguaggio di programmazione Java (livello linguistico 17).
- Framework applicativo: Spring Boot.
- Persistenza e ORM: Spring Data JPA con Hibernate.
- Testing unitario e d'integrazione: JUnit e Mockito.

- **Front-end:**

- Framework JavaScript: Vue.js (eseguito in ambiente di runtime Node.js).
- Strumento di Build e Dev Server: Vite per la compilazione e il bundling ottimizzato.
- Gestione richieste HTTP: libreria Axios per interazione con le API REST.
- Framework grafico: Tailwind CSS per lo stile dell'interfaccia.

- **Database:**

- Database Management System: DBMS relazionale PostgreSQL.

1.7.4 RS-04 Gestione dello Stato e Persistenza

Il Back-end è l'unico responsabile della gestione dello stato del sistema. Tutte le informazioni persistenti relative a utenti, issue, commenti e notifiche, devono essere conservate nel database PostgreSQL dedicato, accessibile esclusivamente dal livello di accesso ai dati del Back-end.

1.7.5 RS-05 Separazione delle Responsabilità

La progettazione del codice sorgente lato Back-end deve favorire la modularità e la manutenibilità attraverso una netta separazione delle responsabilità (Layered Architecture). Devono essere implementati i seguenti livelli logici:

- **Controller:** Per l'esposizione degli endpoint REST e la gestione delle richieste/risposte HTTP;
- **Service:** Per l'isolamento della logica di business dell'applicazione.
- **Repository:** Per l'astrazione e l'esecuzione delle operazioni sul database.
- **DTO (Data Transfer Object):** Per il trasferimento sicuro dei dati tra i vari livelli e verso il client, evitando l'esposizione diretta del modello del database.

1.7.6 RS-06 Meccanismo di Versioning delle Issue

Il Back-end deve implementare il controllo della concorrenza ottimistica tramite un attributo di versione (campo version) associato specificamente all'entità issue. Il sistema deve verificare la corrispondenza della versione prima di effettuare qualsiasi aggiornamento sul database per impedire sovrascritture di dati obsoleti.

1.7.7 RS-07 Ambiente di Deployment e Containerizzazione

L'ambiente di deployment prevede l'isolamento dei servizi per semplificare il rilascio del sistema:

- Il database PostgreSQL deve essere eseguito su un host dedicato, accessibile esclusivamente dal Back-end tramite rete protetta sulla porta 5432.
- I servizi applicativi (Front-end e Back-end) devono essere distribuiti come container Docker indipendenti.

1.7.8 RS-08 Prevenzione Cross-Site Scripting (XSS)

Il livello di presentazione (Front-end) deve mitigare gli attacchi di tipo XSS. Questo comportamento deve essere garantito mediante l'utilizzo del meccanismo automatico data-binding ed escaping dei caratteri speciali offerto dal framework Vue.js, impedendo l'esecuzione di script malevoli inseriti nei campi di input.

1.7.9 RS-09 Prevenzione SQL Injection

Il livello di business (Back-end) non deve comporre query SQL tramite concatenazione diretta di stringhe. La comunicazione con il DBMS relazionale avviene tramite l'ORM Hibernate e le specifiche Spring Data JPA. Le eventuali query native sono comunque definite tramite l'annotazione @Query dei Repository e impiegano esclusivamente parametri associati (binding), mai concatenazione di stringhe: ciò forza l'utilizzo di query parametrizzate, neutralizzando i tentativi di SQL Injection.

1.8 Casi d'uso dettagliati (Modello Cockburn) e relativi mock-up

1.8.1 UC-01 Creazione Issue

USE CASE #1	Creazione Issue		
Goal in Context	Un utente autorizzato segnala una issue.		
Preconditions	L'utente è autenticato nel sistema con il ruolo User.		
Success End Condition	La issue viene salvata nel database, tracciata e inizializzata con <code>version = 0</code> .		
Failed End Condition	La issue non viene salvata; il sistema mostra un errore di validazione dei dati obbligatori.		
Primary Actor	User		
Trigger	L'utente seleziona la funzione "Segnala problema".		
Main Scenario	Step n.	Actor 1 (User)	System
	1	Richiede il modulo di creazione.	Mostra la schermata del form vuoto.
	2	Inserisce Titolo e Descrizione che sono obbligatori.	Permette inserimento del testo.

	3	Seleziona Tipo e Priorità di soluzione.	Mostra i menu a tendina con i rispettivi valori selezionabili.
	4	Invia la segnalazione cliccando sul pulsante.	Inizializza la versione a 0, imposta lo stato su TODO e memorizza l'entità nel DB.
	5		Mostra una conferma visiva di avvenuto salvataggio.
Extension A:	3a	L'utente desidera categorizzare la issue (Etichette)	
	1	Associa una o più etichette testuali prima dell'invio.	Associa l'elenco di etichette alla issue.
Extension B:	4a	Mancanza di campi obbligatori (Titolo)	
	1	Clicca su invia senza compilare il titolo.	Blocca l'invio della richiesta, evidenzia il campo vuoto e mostra un errore.

The mock-up is a dark-themed form titled "Segnala un problema". It contains the following sections:

- TITOLO ***: A text input field with the placeholder "Descrivi brevemente il problema..."
- DESCRIZIONE**: A large text area with the placeholder "Aggiungi dettagli, passi per riprodurre, screenshot..."
- IMMAGINE**: A file upload section with a blue "Choose File" button and the text "No file chosen".
- ETICHETTE**: A text input field with the placeholder "Aggiungi etichetta..." and a blue "+" button to its right.
- TIPO ***: A dropdown menu with the text "Seleziona..." and a downward arrow.
- PRIORITÀ ***: A dropdown menu with the text "Seleziona..." and a downward arrow.
- At the bottom right, there are two buttons: a grey "Annulla" button and a blue "Invia segnalazione" button.

Figura 1.2: Mock-up creazione issue

1.8.2 UC-02 Assegnazione Issue

USE CASE #2	Assegnazione Issue
Goal in Context	Un amministratore delega la risoluzione di una issue a un utente specifico del sistema.

Preconditions	L'amministratore è autenticato; la issue di interesse esiste a sistema, l'utente al quale si desidera assegnare la issue esiste.		
Success End Condition	Il destinatario (campo <code>assegnato_a</code>) viene salvato sulla issue, lo stato passa a <code>IN_PROGRESS</code> e viene generata una notifica.		
Failed End Condition	L'assegnazione fallisce per conflitto di concorrenza ottimistica (stessa issue modificata).		
Primary Actor	Admin		
Trigger	L'amministratore seleziona una issue non assegnata dal pannello operativo.		
Main Scenario	Step n.	Admin	System
	1	Seleziona la issue e apre il menu di assegnazione.	Mostra la schermata di assegnazione.
	2	Inserisce email utente	Abilita il pulsante di conferma dell'operazione.
	3	Conferma l'assegnazione del task.	Incrementa la version, aggiorna il campo <code>assegnato_a</code> nel DB, imposta lo stato su <code>IN_PROGRESS</code> e genera una notifica.
	4		Mostra il messaggio "Assegnazione completata con successo".

Extension A:	2a	L'amministratore vuole imporre una scadenza	
	1	Configura una data limite tramite il selettore temporale.	Registra il timestamp di scadenza legandolo al record della issue.
Extension B:	3a	Concorrenza - Errore di Versioning (RNF-02)	
	1	Tenta di salvare, ma un altro admin ha già modificato la issue.	Rileva la discrepanza del campo <i>version</i> , annulla la transazione, genera errore e notifica l'utente.

Assegna issue #44

DATA DI SCADENZA (OPZIONALE)

mm/dd/yyyy

EMAIL UTENTE A CUI ASSEGNARE

utente@email.com

Annulla Submit

Figura 1.3: Mock-up interfaccia assegnazione issue

2 Documento di Design del Sistema

Il presente capitolo illustra le scelte di progettazione e le soluzioni architetturali adottate per il sistema BugBoard26. L'obiettivo consiste nel mappare i requisiti analizzati precedentemente in componenti macroscopiche coese, definendo la topologia di rete, i vincoli infrastrutturali e l'automazione dei processi di deployment secondo le migliori pratiche dell'ingegneria del sistema.

2.1 Descrizione dell'Architettura Proposta e Criteri di Design

Il sistema BugBoard26 adotta un'architettura distribuita basata sul paradigma Client-Server con una netta separazione tra il livello di presentazione (Front-end) e il livello di logica applicativa (Back-end). Quest'ultimo è internamente strutturato secondo una Layered Architecture (Architettura a Livelli) per segregare le responsabilità logiche. In fase di deployment, il sistema prevede una topologia basata su un Reverse Proxy. Il browser del client non comunica mai direttamente con il server di logica o con il database, ma interagisce esclusivamente con l'istanza di Front-end. Sarà poi il web server del Front-end (Nginx) a instradare le richieste verso i servizi interni sulla rete privata.

2.1.1 Criteri di Design Adottati e Motivazioni

La scelta di questa specifica configurazione risponde a precisi criteri mirati a garantire la qualità del sistema a lungo termine:

1. **Disaccoppiamento Client-Server e Sviluppo Parallelo:** Separando fisicamente il Front-end e il Back-end tramite contratti stabili (API REST), i due ambienti risultano indipendenti. Lato sviluppo, il Front-end può simulare (mockare) le risposte JSON delle API senza attendere il completamento del codice del Back-end, accelerando il ciclo di vita del progetto. Lato produzione, è possibile aggiornare o sostituire l'interfaccia utente senza impattare la logica di business.
2. **Isolamento e Sicurezza della Rete:** Il Back-end e il Database PostgreSQL non necessitano di alcuna esposizione pubblica sulla rete internet. Il Back-end è

raggiungibile solo internamente dal Front-end tramite IP privato all'interno della stessa Virtual Private Cloud (VPC), riducendo drasticamente la superficie di attacco esposta a potenziali minacce esterne.

3. **Risoluzione del CORS a Livello Infrastrutturale (Same-Origin):** Poiché ogni richiesta verso il Back-end viene intercettata dal Reverse Proxy del Front-end sotto il prefisso `/api/`, in produzione l'intera comunicazione risulta same-origin per il browser del client. In questa configurazione la policy CORS (Cross-Origin Resource Sharing) non viene sollecitata, centralizzando la gestione del routing a livello di rete anziché disperderla nella logica applicativa.
4. **Manutenibilità Isolata dei Livelli (Layered):** La strutturazione interna del Back-end in livelli sovrapposti (Presentation, Business, Data Access) fa sì che le modifiche ai requisiti non si propaghino a cascata. Un cambio nelle regole di business toccherà solo il livello di servizio, mantenendo inalterati i canali di comunicazione HTTP e la persistenza.

2.2 Descrizione e Motivazione delle Scelte Tecnologiche

Lo stack tecnologico selezionato per BugBoard26 non rappresenta solo un insieme di strumenti moderni, ma risponde a requisiti di affidabilità, robustezza, scalabilità e automazione.

2.2.1 Infrastruttura Cloud: Google Cloud Platform (GCP)

Il sistema è distribuito su macchine virtuali Compute Engine all'interno di GCP.

- **Motivazione:** GCP garantisce un'infrastruttura di rete globale ad altissime prestazioni e la possibilità di isolare le istanze in una VPC (Virtual Private Cloud). L'uso di IP esterni statici per il Front-end assicura la raggiungibilità costante del servizio, mentre il firewall di GCP permette di blindare le comunicazioni interne (porta 8080 per il Back-end) abilitando solo i canali necessari (SSH per il deploy e HTTP per gli utenti).

2.2.2 Ambiente di Back-end: Java, Spring Boot e PostgreSQL

Per il nucleo elaborativo e la persistenza dei dati si è optato per una soluzione stabile di stampo enterprise:

- **Java 17 e Spring Boot 3.2.5:** Java offre una tipizzazione forte che riduce gli errori a runtime in un sistema critico come la gestione di issue. Spring Boot è stato scelto per il suo paradigma Convention over Configuration, che automatizza la gestione dei

componenti, semplifica l'iniezione delle dipendenze e fornisce un server embedded (Tomcat) efficiente e pronto all'uso.

- **PostgreSQL:** Un DBMS relazionale open source con totale conformità ACID, indispensabile per garantire la consistenza dei dati. PostgreSQL è stato scelto anche per le sue caratteristiche ibride: gestisce nativamente array (text[]) per le etichette delle issue e oggetti strutturati in formato jsonb (usato nel progetto per memorizzare la lista dei commenti nidificati all'interno della issue).
- **Hibernate e Spring Data JPA:** L'adozione di un ORM evita la scrittura di query SQL manuali, astruendo il codice dal dialetto del database e neutralizzando nativamente i tentativi di attacco SQL Injection tramite l'uso sistematico di query parametrizzate.

2.2.3 Ambiente di Front-end: Vue.js 3, Vite, Axios e Tailwind CSS

Il livello di presentazione adotta un approccio Single Page Application (SPA) reattivo ed efficiente:

- **Vue.js 3:** Scelto per il meccanismo di two-way data binding e il Virtual DOM, che permettono di aggiornare l'interfaccia utente in tempo reale non appena variano i dati delle issue. Inoltre, l'escaping automatico nei template previene nativamente gli attacchi di tipo XSS (Cross-Site Scripting).
- **Node.js e Vite:** Node.js opera come ambiente di runtime lato sviluppatore per la gestione dei pacchetti NPM. Vite è stato selezionato come build tool moderno in sostituzione di Webpack: sfrutta i moduli ESM nativi del browser per azzerare i tempi di avvio in sviluppo e utilizza Rollup in fase di build per ottimizzare gli asset tramite tree-shaking, riducendo al minimo il peso del pacchetto scaricato dal client.
- **Nginx e Axios:** Axios gestisce le richieste HTTP asincrone (AJAX) senza ricaricare la pagina web. Nginx serve gli asset statici del Front-end in produzione e funge da Reverse Proxy, gestendo anche il fallback su index.html necessario per l'history mode di Vue, e limitando la dimensione dei caricamenti per consentire l'upload delle immagini allegate alle issue.
- **Tailwind CSS:** Framework grafico che permette di strutturare un'interfaccia utente responsiva direttamente nei componenti, velocizzando la build ed eliminando file CSS globali monolitici.

2.3 Diagramma di Deployment

Al fine di formalizzare la mappatura delle componenti software sui nodi hardware e infrastrutturali descritti, viene introdotto il Diagramma di Deployment UML del sistema. Questo schema evidenzia l'isolamento dei nodi all'interno della Virtual Private Cloud (VPC) e i relativi canali di comunicazione.

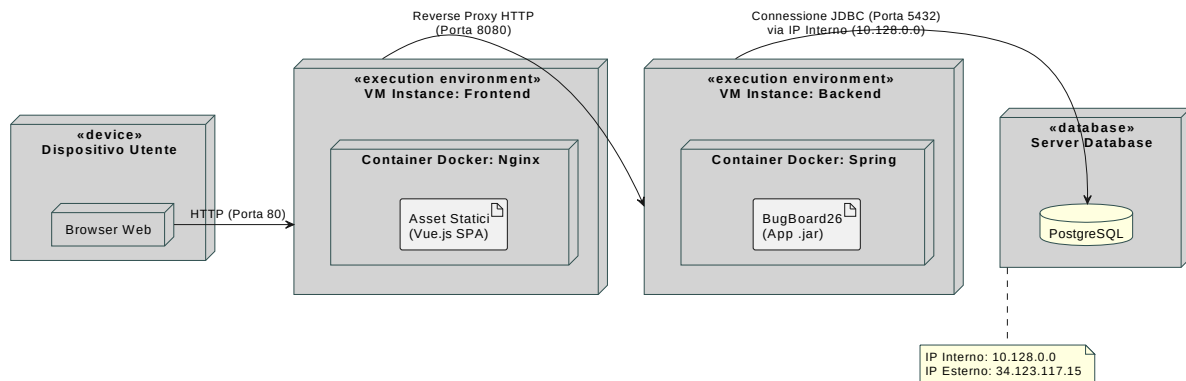


Figura 2.1: Diagramma di Deployment UML dell'infrastruttura di BugBoard26.

Il diagramma si articola sui seguenti nodi:

- **Client Node (Browser dell'utente)**: Avvia la navigazione inoltrando richieste HTTP/HTTPS standard. Non ha visibilità dei nodi interni di backend.
- **VM Istanza Frontend (GCP Compute Engine)**: Esposta pubblicamente su internet. Ospita al suo interno il container Docker di Nginx, che si occupa di servire gli artefatti statici della Single Page Application (Vue.js) ed esegue il re-routing delle richieste `/api/`.
- **VM Istanza Backend (GCP Compute Engine)**: Collocata nella rete privata della VPC. Non possiede un IP pubblico esposto. Ospita il container Docker contenente il runtime Java (Eclipse Temurin 21, bytecode target Java 17) e il framework Spring Boot, comunicando con il Frontend tramite protocollo HTTP interno sulla porta 8080.
- **Database Server (Host PostgreSQL Remoto)**: Nodo di persistenza isolato che accetta connessioni unicamente dall'IP della VM di Backend sulla porta nativa 5432.

2.3.1 Razionale della Distribuzione su Tre Nodi

La scelta di collocare Front-end, Back-end e Database su tre nodi distinti anziché coabitanti sul medesimo host risponde a precise esigenze ingegneristiche:

1. **Isolamento dei domini di guasto:** la saturazione o il crash di un nodo (ad esempio un picco di carico sul Back-end) non si propaga agli altri due; il raggio d'impatto di un guasto resta circoscritto al singolo livello.
2. **Scalabilità indipendente:** i tre livelli hanno profili di carico eterogenei, il Front-end serve asset statici, il Back-end è prevalentemente CPU-bound sulla logica applicativa, il Database è I/O-bound. Nodi separati consentono di dimensionare ciascuno secondo le proprie necessità, senza sovradimensionare gli altri.
3. **Segmentazione della sicurezza (defense in depth):** solo il Front-end è esposto pubblicamente; il Back-end è raggiungibile unicamente dal Front-end e il Database unicamente dal Back-end. Ogni confine tra nodi diventa un punto di controllo presidiato dal firewall, riducendo la superficie d'attacco e impedendo l'accesso diretto al dato persistente.
4. **Ciclo di vita e rilascio indipendenti:** i nodi si aggiornano separatamente, la pipeline prevede due job di deploy distinti per Front-end e Back-end così che il rilascio di una nuova interfaccia non tocchi la logica di business né richieda il fermo del Database.
5. **Specializzazione e manutenzione del nodo dati:** ospitare il Database su un host dedicato permette di configurarlo, aggiornarlo e corredarlo di storage persistente e politiche di backup proprie, indipendentemente dal ciclo di vita dei nodi applicativi, che restano effimeri e ricreabili dall'immagine Docker.

2.4 Strategia di DevOps: Containerizzazione (Docker) e CI/CD (GitHub Actions)

Il ciclo di vita dell'applicazione è interamente automatizzato per garantire la riproducibilità degli ambienti e l'affidabilità dei rilasci:

- **Docker e Multi-stage Build:** Sia per il Front-end che per il Back-end sono stati scritti Dockerfile multi-stage. Questo approccio separa lo stage di compilazione pesante dallo stage di esecuzione minimale (es. JRE per Java e Nginx Alpine per il Front-end), riducendo il peso delle immagini e aumentando la sicurezza.
- **GitHub Actions:** La pipeline (attivata ad ogni push o pull request su main) automatizza l'esecuzione dei test unitari e la build del Front-end, compila le immagini Docker e le pubblica sul registro privato GHCR (GitHub Container Registry). Nei soli push su main, la pipeline aggiorna in parallelo i due server (Front-end e Back-end) su GCP tramite connessioni SSH, la cui chiave privata è conservata codificata in Base64 nei GitHub Secrets, azzerando le procedure di deployment manuale.

3 Design del Software

Il presente capitolo descrive in dettaglio la progettazione interna e l'architettura logica delle componenti software del sistema BugBoard26. Viene approfondito il design del Back-end, analizzando la scomposizione in livelli, lo schema di persistenza, i diagrammi UML strutturali e l'applicazione dei design pattern e dei criteri di qualità del codice.

3.1 Architettura Generale a Livelli (Layered Architecture)

Il sistema Back-end è stato costruito seguendo il pattern architetturale Layered Architecture (Architettura a Livelli). Tale approccio garantisce la separazione delle responsabilità e riduce l'accoppiamento tra le componenti. L'applicazione è suddivisa in quattro livelli logici sovrapposti:

- **Controller Layer (Presentation):** Rappresenta il punto di ingresso delle richieste HTTP provenienti dal Front-end. Intercetta i dati in formato JSON/Multipart, estrae l'identificativo di sessione dall'header X-Session-Id e delega l'elaborazione al livello di business che effettua i controlli di sessione e di ruolo restituendo risposte strutturate tramite DTO profilati.
- **Service Layer (Business Logic):** Contiene la logica applicativa e di business del sistema, inclusi i controlli di sessione e di ruolo, e manipola le entità di dominio. Non gestisce direttamente richieste e risposte HTTP (opera su identificativi e DTO) e accede alla persistenza unicamente attraverso l'astrazione dei Repository.
- **Repository Layer (Data Access):** Fornisce l'astrazione per l'accesso alla persistenza dei dati. Sfrutta Spring Data JPA per interfacciare l'applicazione con il database PostgreSQL, traducendo le operazioni sugli oggetti in query SQL.
- **Domain/Model Layer (Data Core):** Rappresenta il modello dei dati puro dell'applicazione, costituito dalle entità JPA mappate direttamente sulle tabelle del database relazionale.

Oltre ai quattro livelli, due meccanismi trasversali completano l'architettura preservando la separazione delle responsabilità.

3.1.1 Gestione Centralizzata degli Errori

La traduzione degli errori applicativi in risposte HTTP è isolata in un unico punto, così che il livello di business non manipoli mai status code o payload di rete. Tutte le eccezioni di dominio derivano dalla classe astratta `ApiException`, che incapsula lo `HttpStatus` corrispondente: `NotFoundException` → 404, `ForbiddenException` → 403, `ConflictException` → 409, `BadRequestException` → 400. Un componente annotato con `@RestControllerAdvice` (`GlobalExceptionHandler`) intercetta tali eccezioni e costruisce la risposta leggendo lo status direttamente dall'eccezione; gestisce inoltre le eccezioni di lock ottimistico sollevate da Hibernate, convertendole in un 409 Conflict. L'aggiunta di un nuovo tipo di errore si riduce così alla definizione di una sottoclasse, senza modificare né i Service né l'handler.

Listing 3.1: Gestore centralizzato che traduce le eccezioni di dominio in risposte HTTP.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // Le ApiException portano con se' il proprio HttpStatus.
    @ExceptionHandler(ApiException.class)
    public ResponseEntity<Map<String, String>> handleApi(ApiException ex) {
        return ResponseEntity.status(ex.getStatus())
            .body(Map.of("message", ex.getMessage()));
    }

    // Il lock ottimistico di Hibernate diventa un 409.
    @ExceptionHandler({
        ObjectOptimisticLockingFailureException.class,
        OptimisticLockingFailureException.class,
        OptimisticLockException.class
    })
    public ResponseEntity<Map<String, String>> handleOptimisticLock(Exception ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(Map.of("message", "Issue aggiornata da un altro utente:
ricarica e riprova."));
    }
}
```

3.1.2 Flusso di Autenticazione tra i Livelli

L'autenticazione è stateful e attraversa i livelli in modo coerente con la separazione delle responsabilità. Al login il sistema crea una Sessione con chiave sid di tipo UUID e la restituisce al client. Nelle richieste successive il client trasmette il sid nell'header HTTP X-Session-Id (mai nell'URL, per evitarne la comparsa nei log del server e nella cronologia del browser). Il Controller si limita a estrarre l'header; è il Service che, tramite ISessionService, risolve il sid nell'utente associato e ne verifica il ruolo prima di autorizzare l'operazione. La logica di autorizzazione resta così concentrata nel livello di business e indipendente dal meccanismo di trasporto della sessione.

3.1.3 Ottimizzazioni del Livello di Accesso ai Dati

Il livello di accesso ai dati adotta due accorgimenti per contenere il costo delle interrogazioni.

Proiezione diretta su DTO. Le viste a elenco non necessitano del contenuto binario degli allegati. Per evitarne il caricamento, i Repository impiegano constructor expression JPQL che selezionano i soli campi necessari e istanziano direttamente il DTO: la colonna immagine (bytea) non viene mai letta dal database e non vengono materializzate entità gestite, restituendo oggetti già pronti per la serializzazione (cfr. il metodo `findAllIssuesSenzaImmagine`).

Aggiornamento incrementale per versione. La dashboard amministrativa si mantiene aggiornata senza ricaricare l'intera lista: un'interrogazione leggera (`findAllVersions`) restituisce le sole coppie (id, version). Il client confronta periodicamente tali versioni con quelle già in proprio possesso e richiede il ricaricamento di nuovo come DTO privo di immagine delle sole issue nuove o modificate, rilevando inoltre le eventuali rimozioni. In questo modo il campo version, introdotto per il controllo di concorrenza ottimistica (RS-06), viene riutilizzato come rilevatore di cambiamenti a basso costo.

3.2 Descrizione dello Schema per la Persistenza Dati

La persistenza dei dati del sistema BugBoard26 si appoggia sul DBMS relazionale PostgreSQL. L'astrazione dello storage e la mappatura oggetto-relazionale (ORM) sono affidate a Hibernate tramite le specifiche Jakarta Persistence (JPA). Al fine di formalizzare la struttura delle tabelle, dei vincoli di integrità referenziale (Foreign Key) e dei vincoli di

unicità, viene introdotto lo schema logico del database.

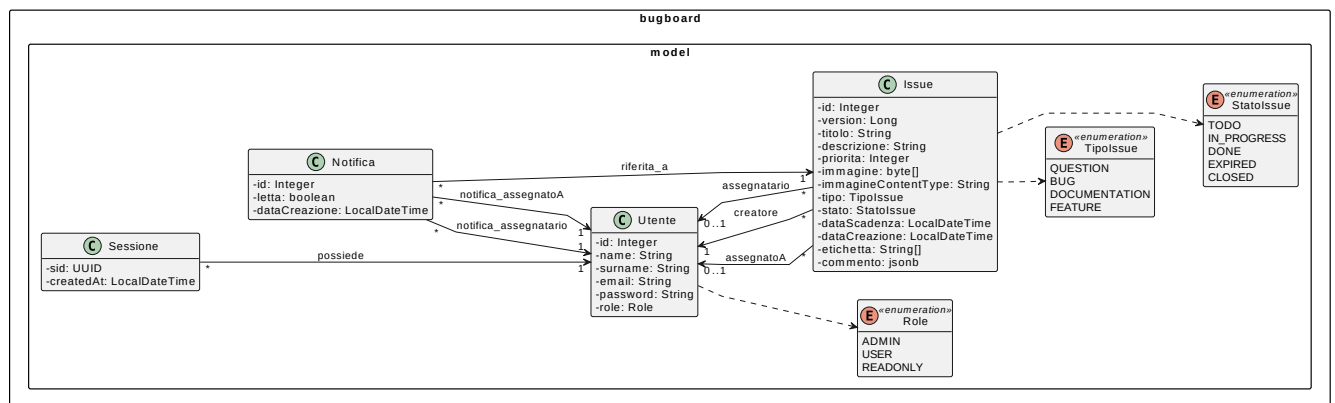


Figura 3.1: Diagramma Entity-Relationship dello schema di persistenza di BugBoard26.

Le entità del dominio e le relative tabelle sono strutturate come segue:

3.2.1 Entità Utente

Mappa gli account degli operatori del sistema. Ad ogni utente è associato un ruolo specifico che governa la sicurezza degli accessi a livello applicativo.

- **id** (Integer, Primary Key, IDENTITY)
- **email** (String, Unique, Nullable=False)
- **password** (String, Nullable=False, codificata tramite hashing forte BCrypt)
- **name** e **surname** (String, Nullable=False)
- **role** (Enum Role: ADMIN, USER, READONLY) mappato come tipo nativo PostgreSQL `utenteruolo`.

3.2.2 Entità Issue

Rappresenta il nucleo informativo dell'applicazione.

- **id** (Integer, Primary Key, IDENTITY)
- **version** (Long, annotato @Version) per il controllo della concorrenza ottimistica (cfr. RS-06); inizializzato a 0 alla prima persistenza e incrementato a ogni aggiornamento.
- **titolo** (varchar, Nullable=False) e **descrizione** (TEXT, Nullable=False)
- **priorita** (Integer, opzionale)
- **immagine** (bytea) e **immagine_content_type** (String) per la persistenza di allegati multimediali.

- **tipo** (Enum TipoIssue: QUESTION, BUG, DOCUMENTATION, FEATURE) mappato come tipo nativo PostgreSQL `issuetipo`.
- **stato** (Enum StatoIssue: TODO, IN_PROGRESS, DONE, EXPIRED, CLOSED) mappato come tipo nativo PostgreSQL `issuestato`, con default TODO.
- **datascadenza** (timestamp, opzionale) e **datacreazione** (timestamp, valorizzato dal database).
- **etichetta** (text[]) mappato come array nativo PostgreSQL per una gestione flessibile dei tag.
- **commento** (jsonb) che sfrutta i vantaggi dei database ibridi persistendo una lista strutturata di oggetti Commento (testo, timestamp, autore) direttamente in un campo JSON binario ad alte prestazioni, evitando la saturazione di tabelle di join per dati puramente consultivi.
- **Relazioni e Vincoli di Integrità:**
 - **idcreatore** → Associazione Many-To-One (foreign key) verso l'utente che ha creato la issue.
 - **idassegnatario** → Associazione Many-To-One (foreign key) verso l'utente con ruolo ADMIN che ha effettuato l'assegnazione prendendo in carico la issue.
 - **assegnato_a** → Associazione Many-To-One (foreign key) verso l'utente con ruolo USER (destinatario) incaricato della risoluzione.

3.2.3 Entità Sessione e Notifica

- **Sessione:** Gestisce lo stato dell'autenticazione tramite accoppiamento di sicurezza. Utilizza una chiave primaria non sequenziale `sid` di tipo UUID (Universally Unique Identifier) per prevenire attacchi di ID-enumeration sulla rete, legata con vincolo di chiave esterna all'utente associato.
- **Notifica:** Memorizza gli eventi di assegnazione delle issue per aggiornare la dashboard degli utenti. Relaziona in modalità Many-To-One la singola issue, l'assegnatario (admin) e il destinatario (user), e mantiene un flag `letta` (boolean) e un timestamp `datacreazione`.

3.3 Diagramma delle Classi di Design

Il Diagramma delle Classi di Design formalizza la struttura logica del codice sorgente del Back-end, evidenziando le relazioni di dipendenza, ereditarietà e associazione tra le componenti dei diversi livelli (Controllers, Services, Repositories, Entities).

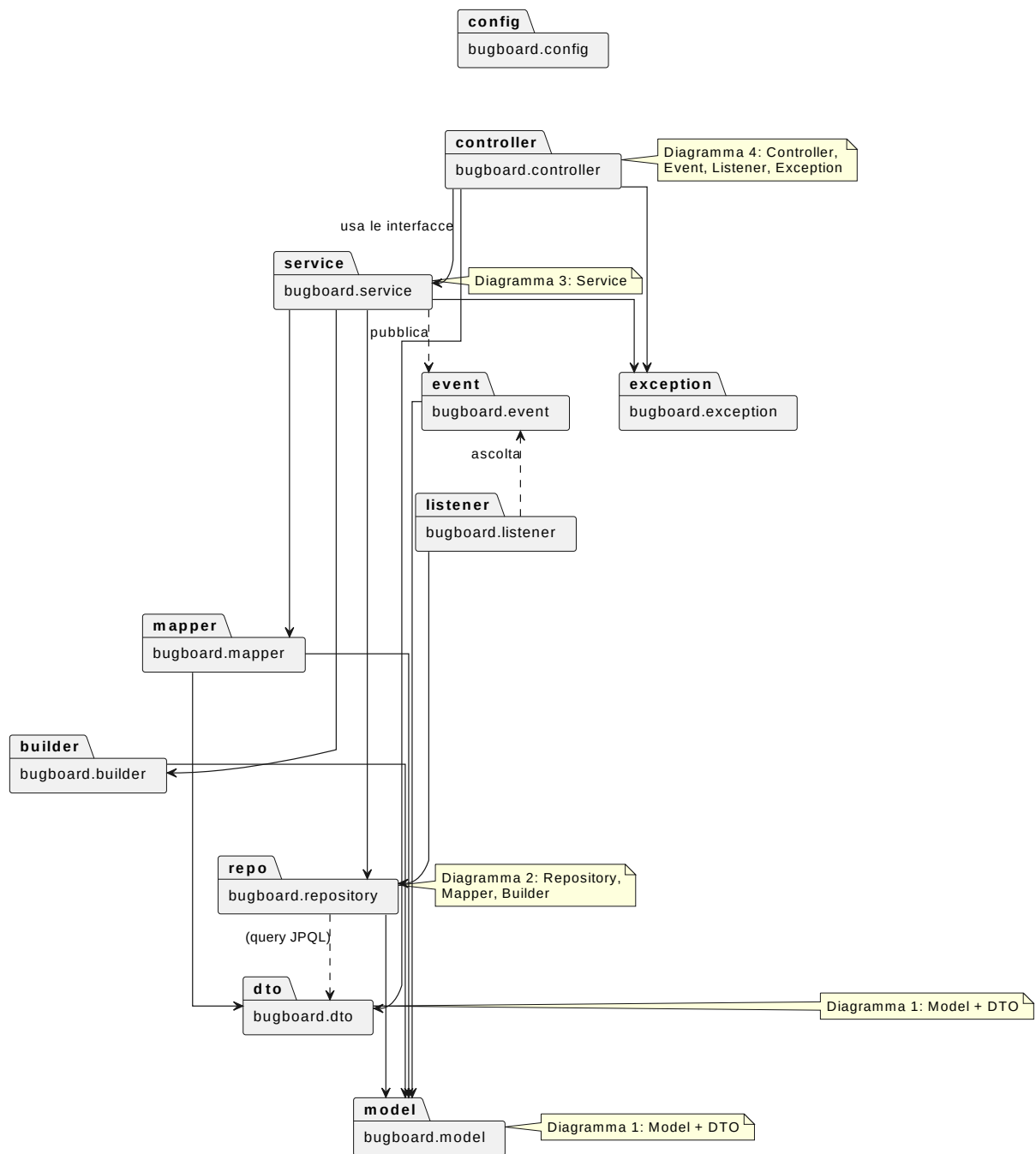


Figura 3.2: Vista dei vari package e delle loro dipendenze

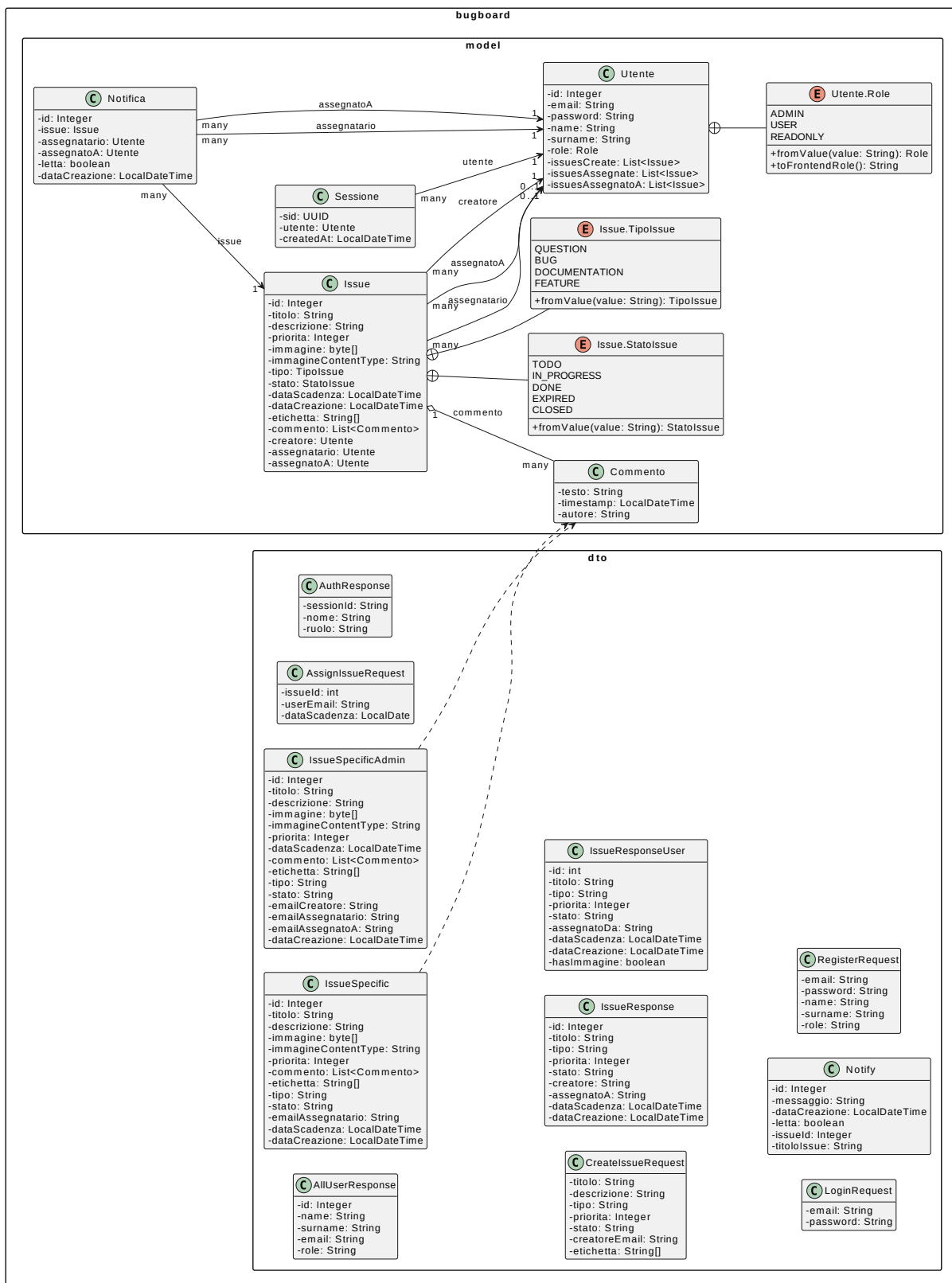


Figura 3.3: Model e DTO del class diagram di design

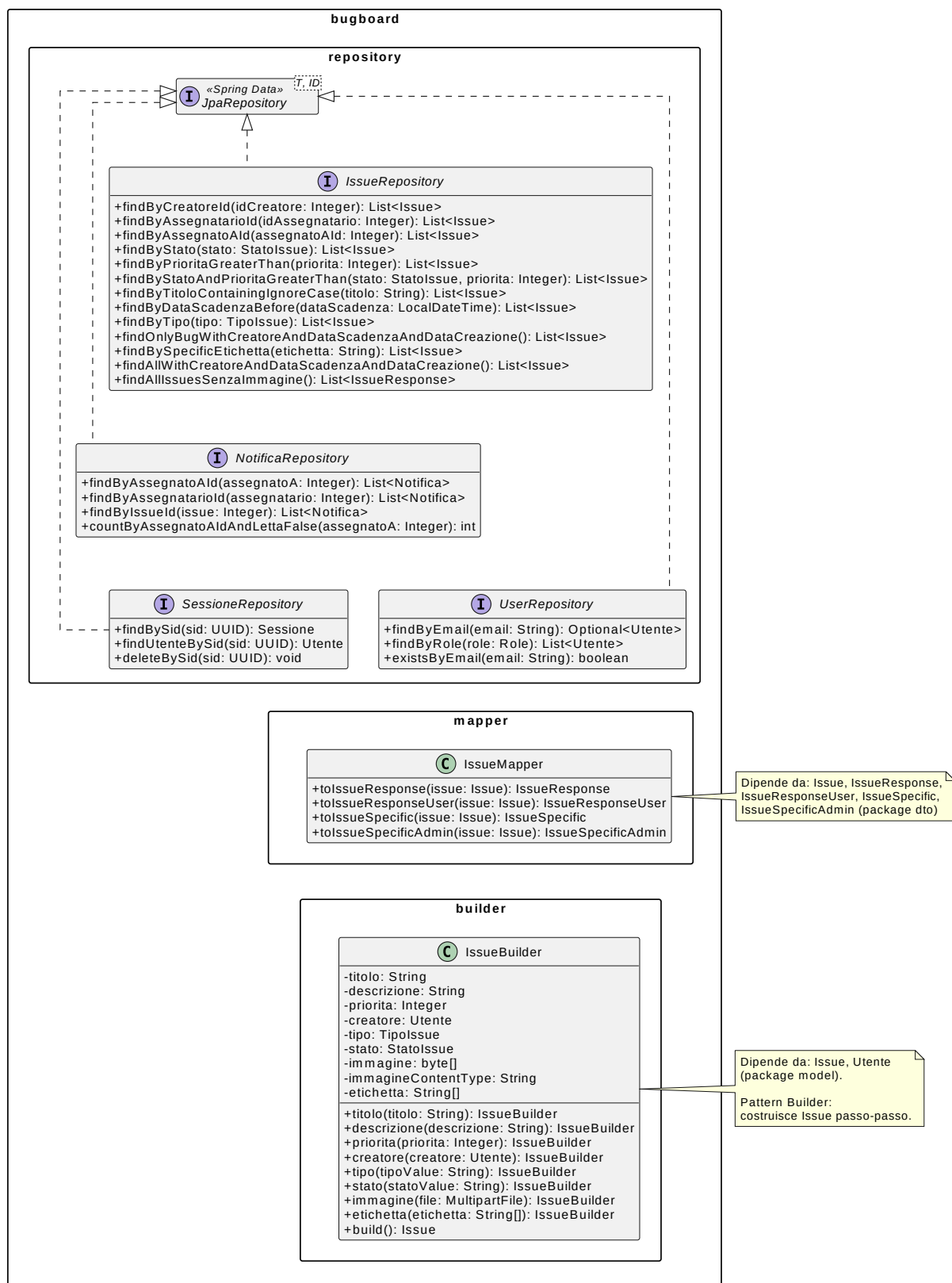


Figura 3.4: Repository, mapper e builder del class diagram di design

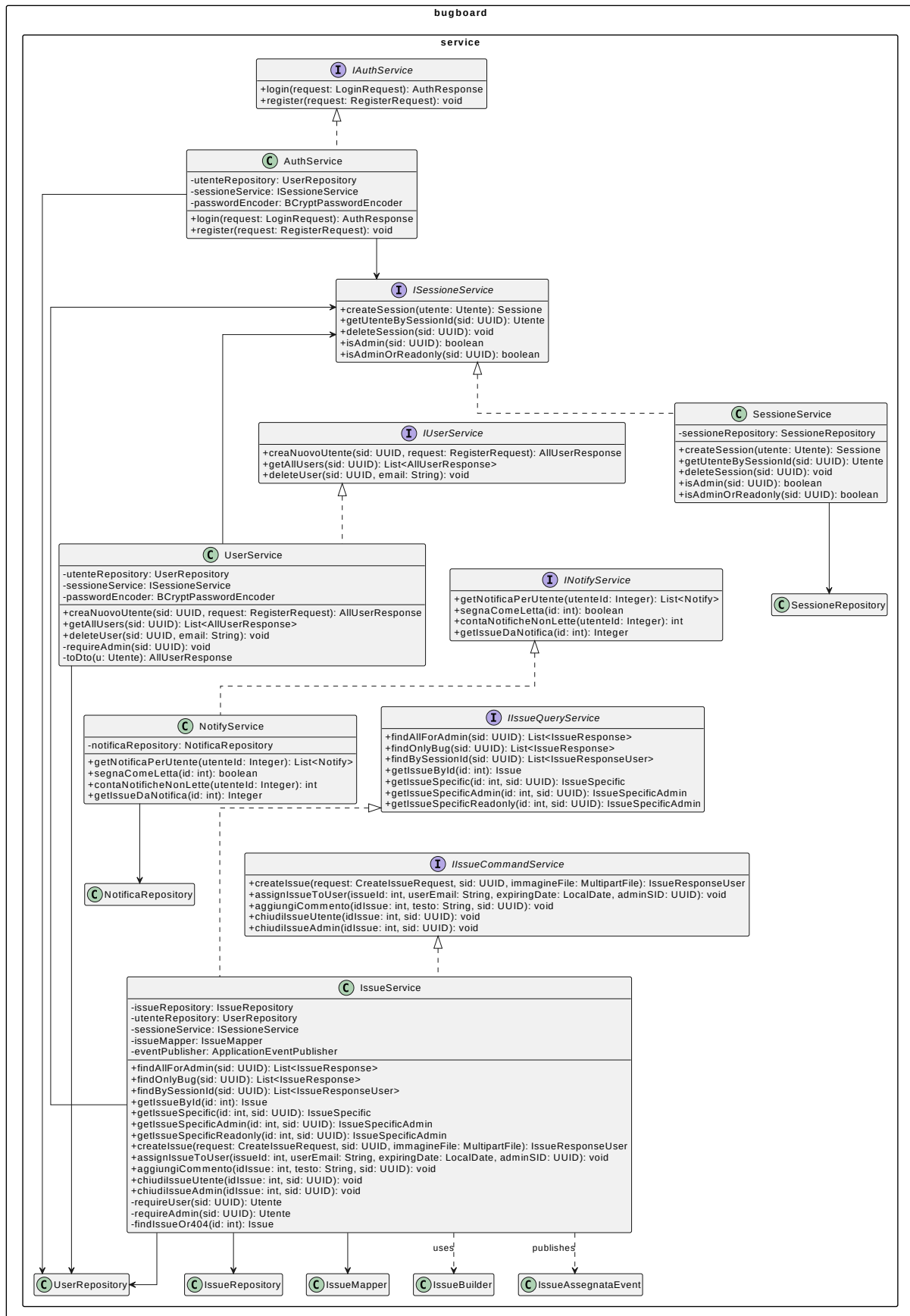


Figura 3.5: Service del class diagram di design

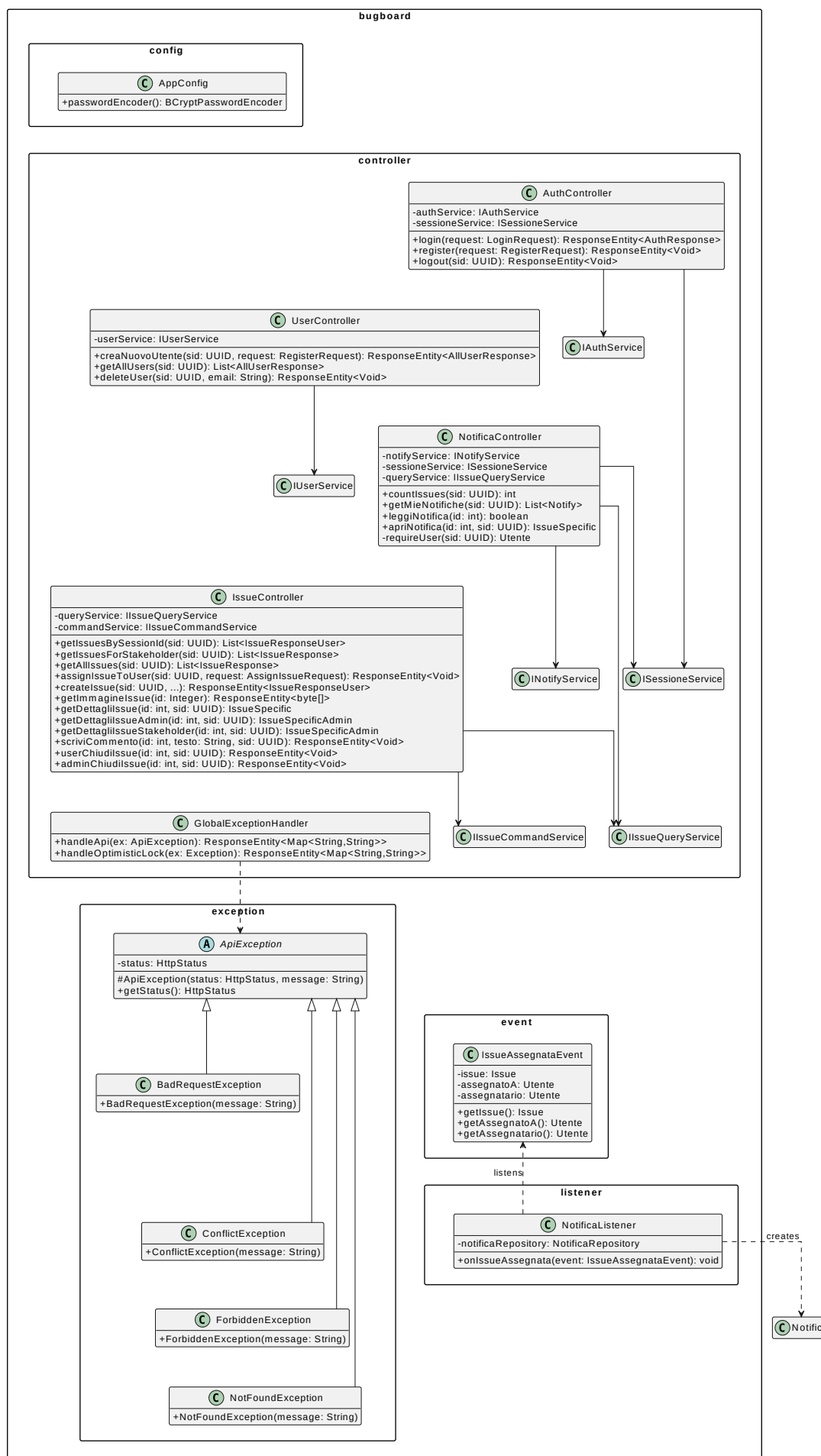


Figura 3.6: Controller, Event/Listener e Exception del class diagram di design

Il codice è organizzato sotto il package radice `bugboard` (coordinate Maven: `groupId com.bugboard`, `artifactId backend`). I sotto-package seguono gli standard architetturali del framework Spring Boot:

- `bugboard.controller`: Contiene le classi REST Controller esposte pubblicamente, deputate alla mappatura degli endpoint e alla validazione formale dell'input HTTP.
- `bugboard.service`: Racchiude le interfacce di business e le relative implementazioni transazionali, dove risiede l'effettiva logica applicativa del sistema.
- `bugboard.repository`: Interfacce che estendono `JpaRepository` per l'interrogazione automatizzata dello schema di persistenza.
- `bugboard.model`: Classi di dominio annotate come entità JPA persistenti.
- `bugboard.dto`: Oggetti leggeri usati per disaccoppiare la rappresentazione esterna dei dati dal modello interno del database.
- `bugboard.builder`, `bugboard.mapper`: Componenti di supporto per la costruzione delle entità `Issue` e la conversione `entità ↔ DTO`.
- `bugboard.event`, `bugboard.listener`: Infrastruttura event-driven per la generazione disaccoppiata delle notifiche.
- `bugboard.exception`, `bugboard.config`: Eccezioni applicative (con gestore centralizzato) e configurazione dei bean.

3.4 Descrizione e Motivazione delle Scelte di Software Design

Il software design si distingue per l'applicazione rigorosa dei design pattern e dei principi SOLID, mirati a soddisfare i requisiti non funzionali di estensibilità, manutenibilità e isolamento dei componenti:

3.4.1 Pattern Builder

La creazione di un'entità complessa come `Issue` viene interamente segregata nella classe specializzata `IssueBuilder`. Questo pattern offre diversi vantaggi:

- Agisce come accentratore della logica di conversione dai campi stringa provenienti dai DTO verso i tipi fortemente tipizzati del dominio (`Enum`).
- Gestisce in modo incapsulato le eccezioni di I/O durante la conversione dei file allegati in array di byte, ripulendo il Service da blocchi try-catch disordinati.

- Previene la violazione dello stato dell'oggetto, impedendo l'istanza di entità inconsistenti o parzialmente configurate (l'unico modo per ottenere la issue è invocare `build()`).

3.4.2 Pattern Observer e Disaccoppiamento Event-Driven

Al fine di non violare il principio di singola responsabilità SRP (Single Responsibility Principle), la logica di generazione e invio delle notifiche agli utenti è stata completamente disaccoppiata dalle funzioni del business layer delle issue. Quando un amministratore assegna una issue:

1. Il metodo transazionale in `IssueService` esegue la persistenza dello stato aggiornato dell'issue.
2. Tramite l'infrastruttura `ApplicationEventPublisher` di Spring, viene sollevato un evento `IssueAssegnataEvent`.
3. Un componente isolato, denominato `NotificaListener`, si pone in ascolto dell'evento tramite l'annotazione `@EventListener` e provvede in totale autonomia a creare e registrare la notifica sul DB.

Grazie a questo approccio, `IssueService` non possiede alcuna dipendenza verso `NotificaRepository`, azzerando l'accoppiamento strutturale tra chi genera l'azione e chi reagisce ad essa.

3.4.3 Pattern Data Transfer Object (DTO) e Mappers

Per impedire la sovraesposizione del modello dati interno e ottimizzare il payload di rete, le entità non transitano mai direttamente verso il network HTTP. Il livello di presentazione scambia esclusivamente DTO profilati per ruolo e caso d'uso. La conversione dell'entità `Issue` verso i diversi DTO è interamente delegata al componente specializzato `IssueMapper`, isolando la Separation of Responsibilities.

3.4.4 Dependency Inversion Principle (DIP) e Inversione del Controllo (IoC)

Tutte le collaborazioni strutturali avvengono dipendendo esclusivamente da interfacce e astrazioni e mai da implementazioni concrete. Ad esempio, i REST Controller invocano le interfacce `IIssueCommandService`, `IIssueQueryService` e `ISessionService`. L'istanza, il ciclo di vita e l'iniezione dei componenti (eseguita tramite Constructor Injection) sono completamente delegati al container IoC di Spring Framework, facilitando l'introduzione di modifiche o l'iniezione di mock in fase di testing.

3.4.5 Interface Segregation e Separazione Comando/Interrogazione

Le operazioni sulle issue sono esposte attraverso due interfacce distinte, `IssueQueryService` per le sole letture e `IssueCommandService` per le sole scritture, entrambe implementate dall'unica classe `IssueService`. La suddivisione separa per contratto le operazioni che modificano lo stato da quelle di sola consultazione (Command-Query Separation) e rispetta l'Interface Segregation Principle (ISP): un componente può dipendere dal solo sottoinsieme di operazioni che gli serve, anziché da un'unica interfaccia ampia. La distinzione rende inoltre più mirato il testing, poiché consente di sostituire con mock separatamente le letture e le scritture.

3.5 Evidenza dell'Uso di Strumenti di Versioning

Lo sviluppo del sistema BugBoard26 è stato interamente tracciato mediante l'uso del sistema di controllo versione distribuito Git, appoggiandosi alla piattaforma cloud GitHub per la gestione del repository remoto, l'integrazione del codice e la collaborazione attiva tra i membri del team. Il codice sorgente e la cronologia completa dei rilasci sono accessibili al seguente indirizzo:

- **Link Repository GitHub:** <https://github.com/rccmolinari/BugBoard2026.git> [Repository Pubblica]

Al fine di garantire la trasparenza dello sviluppo e documentare la continuità del ciclo di vita del software, vengono di seguito riportati e analizzati i dati statistici complessivi estratti direttamente dagli strumenti di analitica integrati nella piattaforma GitHub (GitHub Insights). I dati coprono l'intero arco temporale di attività del progetto, estendendosi dal primo commit di aprile 2026 fino al completamento dei rilasci a metà luglio 2026.

3.5.1 Analisi dei Commit e Frequenza Temporale

Come evidenziato dal report cronologico annuale dei commit, l'attività sul repository si concentra interamente nel quadrimestre finale del progetto (aprile-luglio 2026), registrando un totale complessivo di 98 commit (escludendo i commit ci/cd). L'andamento grafico mostra una progressione strutturata secondo precise fasi di sviluppo e incremento funzionale:

- **Fase di Inizializzazione (Seconda metà di Aprile 2026):** Il progetto prende il via nella seconda metà di aprile con i primi 6 commit di setup, seguiti da una breve fase di pianificazione (1 commit nell'ultima settimana del mese).

- **Fase di Picco Architettuale (Maggio 2026):** Il mese di maggio rappresenta il fulcro dello sviluppo e della stesura delle funzionalità core del sistema. Si registra un’impennata iniziale con 17 commit, culminando nella seconda settimana di maggio con il picco massimo assoluto dell’intero progetto, pari a 28 commit in soli sette giorni. Dopo una breve flessione (2 commit), l’attività riprende intensamente a fine mese con ulteriori 12 commit.
- **Fase di Consolidamento e Rifinitura (Giugno-Luglio 2026):** il mese di giugno si apre con 5 commit dedicati alla stabilizzazione dei moduli. Dopo una breve pausa centrale, l’ultima decade del mese vede uno sprint finale di chiusura con 3 commit e infine 13 commit nell’ultima settimana di giugno; questi ultimi sono incentrati principalmente sul bug-fixing, sulla rifinitura dell’interfaccia e sul rilascio delle componenti finali. Nella prima metà di luglio, infine, vengono effettuati i commit conclusivi riguardanti il caricamento della documentazione.

3.5.2 Contributori e Ripartizione del Lavoro

Il codice sorgente è stato sviluppato in modo collaborativo da due contributori principali, i quali hanno operato in sinergia coprendo sia la parte infrastrutturale che quella applicativa. La ripartizione quantitativa del lavoro e l’impatto complessivo sul codebase sono così dettagliati:

1. **rccmolinari (Contributore #1):** Ha effettuato un totale di 52 commit. Il suo apporto si distingue per una massiccia introduzione e modifica di righe di codice, mantenendo un’attività distribuita lungo tutto l’arco temporale del progetto con costanza visibile anche nelle ultime fasi di giugno.
2. **daniele107 (Contributore #2):** Ha effettuato un totale di 46 commit. La sua attività si è concentrata principalmente nella prima metà del ciclo di sviluppo (con un picco evidente tra fine aprile e la metà di maggio) e nella parte iniziale di luglio.

L’equilibrio numerico nella distribuzione dei commit (52 contro 46) evidenzia una cooperazione bilanciata e sinergica nella gestione del flusso di lavoro, garantendo una conoscenza condivisa del codice e una progressione fluida dei requisiti del sistema.

3.6 Report di Qualità del Codice

Al fine di garantire la manutenibilità a lungo termine del software, l’assenza di vulnerabilità di sicurezza latenti e il rispetto degli standard di codifica della comunità Java, il modulo Back-end del sistema è stato sottoposto ad analisi statica del codice (Static Application Security Testing - SAST). L’analisi è stata condotta mediante l’integrazione

della piattaforma SonarQube all'interno del ciclo di build. Nel presente paragrafo verranno riportate le metriche estratte dalla dashboard di qualità, con particolare focus su:

- **Code Smells e Debito Tecnico:** Valutazione dello sforzo necessario per la ristrutturazione del codice non ottimale.
- **Duplicazione del Codice:** Percentuale di linee di codice duplicate all'interno dei package.
- **Bugs e Vulnerabilità:** Evidenza dell'assenza di falle di sicurezza o di memory leak.

Nel presente report vengono integrati e commentati i risultati emersi dall'analisi statica eseguita sul progetto bugboard26. L'istanza analizzata è composta da circa 2.2k linee di codice sorgente Java. La piattaforma SonarQube ha promosso il codice con esito Passed (Quality Gate superato con successo). Di seguito si riporta lo specchietto riassuntivo delle metriche chiave rilevate sulla dashboard:

Tabella 3.1: Metriche di Qualità estratte dalla Dashboard SonarQube

Ambito d'Analisi	Metrica Rilevata	Rating Conseguito
Security	0 Vulnerabilità / Security Issues	A
Reliability	2 Bugs minori	A
Maintainability	4 Code Smells	A
Duplications	0.0% Linee duplicate	-
Coverage	0.0% (Test eseguiti localmente) ¹	-

3.6.1 Analisi dei Risultati e Debito Tecnico

I dati estratti evidenziano un profilo qualitativo estremamente alto, come testimoniato dal triplo rating A ottenuto nelle categorie principali:

- **Sicurezza ed Affidabilità:** Non è stata riscontrata alcuna vulnerabilità di sicurezza latente (0 Security Issues). I 2 Bugs evidenziati rientrano nella categoria delle anomalie minori di affidabilità e non compromettono la stabilità a runtime dell'applicazione.
- **Manutenibilità e Pulizia:** La presenza di soli 4 Code Smells (violazioni minori degli standard di codifica) si traduce in un debito tecnico quasi nullo, quantificabile in pochi minuti di refactoring. La percentuale di Duplicazione del Codice è pari allo 0.0%, confermando un'eccellente modularità del codice e il pieno rispetto del principio DRY (Don't Repeat Yourself).

3.6.2 Dettaglio dell’Analisi Complessiva (Overall Code)

La vista della dashboard estesa fornisce un quadro consolidato sull’intero ciclo di vita dell’applicazione alla versione 1.0, dettagliando il perimetro del codice esaminato e confermando l’affidabilità delle metriche aggregate.

Tabella 3.2: Indicatori di Configurazione ed Elementi di Copertura

Metrica / Indicatore	Valore / Stato Rilevato
Versione Applicativa	Version 1.0
Linee di Codice Totali	2.2k Lines of Code
Linee Computate per Duplicazione	2.2k Lines complessive analizzate
Linee da Coprire (Jacoco Target)	427 linee totali da sottoporre a test
Security Hotspots	0 (Rating A)
Accepted Issues	0 (Nessuna anomalia strutturale accettata o derogata)

Attraverso questo spaccato, è possibile constatare che:

- **Assenza di Hotspot di Sicurezza:** Il contatore Security Hotspots pari a 0 indica che nessuna porzione di codice espone pattern o costrutti tipicamente associati a rischi di sicurezza (es. algoritmi di cifratura deboli, hardcoded credentials, o errata gestione dei flussi I/O).
- **Rigore nel Quality Gate:** Il valore di 0 Accepted Issues certifica che il superamento del Quality Gate (Passed) non è stato viziato dall’esclusione manuale o dal mascheramento di anomalie bloccanti da parte dello sviluppatore, il codice rispetta gli standard in modo nativo.
- **Perimetro di Coverage:** Le 427 linee identificabili rappresentano la logica decisionale e di business core effettiva (classi di servizio e controller), escludendo dal conteggio boilerplate code automatico, interfacce o DTO, confermando l’adeguatezza dell’architettura per un futuro tracciamento automatico della copertura.

3.7 Codice Sorgente Sviluppato e Struttura di Build

Il codice sorgente completo del sistema è strutturato all’interno di un unico repository suddiviso nei due macro-moduli principali (backend e frontend). L’automazione della compilazione e dell’impacchettamento del software è governata in modo dichiarativo tramite descrittori di build standardizzati. La gestione delle dipendenze e la compilazione automatica del modulo Back-end sono interamente definite all’interno del file pom.xml di Maven, che specifica le librerie di Spring Boot, i driver PostgreSQL e i plugin per la generazione dei container. Il modulo Front-end è orchestrato tramite il descrittore package.json di Node.js e configurato per la build ottimizzata via Vite. Entrambi i moduli

contengono i rispettivi descrittori Dockerfile multi-stage, atti a standardizzare l'ambiente di runtime e a garantire l'esatta riproducibilità del software sia in ambiente locale tramite Docker Compose, sia sui server di produzione distribuiti su Google Cloud Platform.

3.7.1 Descrittore di Build Back-end: pom.xml

La gestione delle dipendenze, dei plugin di compilazione e delle versioni del framework Spring Boot 3.2.5 e del livello linguistico Java 17 è interamente centralizzata nel file di configurazione di Maven:

Listing 3.2: File pom.xml per l'automazione della build del Back-end Spring Boot.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org
  /2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.
  org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
  </parent>

  <groupId>com.bugboard</groupId>
  <artifactId>backend</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>BugBoard Backend</name>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>org.springframework.security</groupId>
        <artifactId>spring-security-crypto</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>
```

3.7.2 Containerizzazione del Back-end: backend/Dockerfile

Il ciclo di build dell'immagine Docker per il Back-end adotta un approccio multi-stage, isolando lo strato delle dipendenze per velocizzare le compilazioni successive e applicando

policy di sicurezza basate su utente non-root:

Listing 3.3: Dockerfile multi-stage per il microservizio Back-end.

```
# syntax=docker/dockerfile:1

# --- Stage 1: build del jar -----
# JDK 21 fisso: Lombok 1.18.32 non compila su JDK 25 (vedi note di progetto).
# Il pom ha <java.version>17>, quindi il bytecode prodotto e' Java 17.
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app

# Prima solo il pom: lo strato delle dipendenze si ricostruisce solo se cambia
# il pom.
COPY pom.xml .
RUN mvn -B -ntp dependency:go-offline

# Poi i sorgenti. I test girano nella pipeline CI, qui li saltiamo per velocita'.

COPY src ./src
RUN mvn -B -ntp clean package -DskipTests

# --- Stage 2: runtime -----
FROM eclipse-temurin:21-jre AS runtime
WORKDIR /app

# Utente non-root
RUN groupadd -r spring && useradd -r -g spring spring
USER spring

# Solo l'eseguibile repackaged (il glob esclude il .jar.original di Spring Boot).

COPY --from=build /app/target/*.jar app.jar

# Config via env: nessun segreto e' baked nell'immagine.
ENV SPRING_PROFILES_ACTIVE=docker
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0"

EXPOSE 8080
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
```

3.7.3 Descrittore di Build Front-end: package.json

Il modulo di presentazione definisce in modo deterministico le dipendenze di produzione (Vue 3, Vue Router, Axios) e gli strumenti di ottimizzazione grafica e di compilazione gestiti tramite Vite:

Listing 3.4: File package.json per la gestione dell'ambiente Front-end.

```
{
  "name": "progettoingsw-vue",
  "private": true,
  "version": "0.0.1",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.16.0",
    "vue": "^3.4.0",
    "vue-router": "^5.0.4"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "^5.0.0",
    "autoprefixer": "^10.4.17",
    "postcss": "^8.4.33",
    "tailwindcss": "^3.4.1",
    "vite": "^5.0.0"
  }
}
```

3.7.4 Containerizzazione del Front-end: frontend/Dockerfile

L'ambiente Front-end compila gli asset tramite un ambiente Node.js isolato e affida la distribuzione degli elementi statici a un'istanza leggera di Nginx, configurata per agganciare dinamicamente l'URL di back-end:

Listing 3.5: Dockerfile per la build e la distribuzione web tramite Nginx.

```
# syntax=docker/dockerfile:1

# --- Stage 1: build statico Vite -----
FROM node:24-alpine AS build
```

```
WORKDIR /app

# npm ci usa il package-lock.json -> build riproducibile.
COPY package.json package-lock.json ./
RUN npm ci

COPY . .
RUN npm run build

# --- Stage 2: nginx che serve la SPA e fa da reverse proxy -----
FROM nginx:1.27-alpine AS runtime

# Sostituisco SOLO ${BACKEND_URL} nel template; cosi' le variabili runtime di
# nginx ($host, $uri, ...) NON vengono toccate da envsubst all'avvio.
ENV NGINX_ENVSUBST_FILTER=BACKEND_URL
ENV BACKEND_URL=http://backend:8080

COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf.template /etc/nginx/templates/default.conf.template

EXPOSE 80
```

3.7.5 Orchestrazione Locale Multi-Container: docker-compose.yml

Al fine di consentire l'esecuzione integrata, locale e deterministica dell'intero stack applicativo riproducendo fedelmente la topologia di rete isolata cloud (evitando la configurazione di eccezioni CORS a livello applicativo), viene utilizzato il file di orchestrazione multi-container seguente:

Listing 3.6: File docker-compose.yml per la simulazione e il testing dell'ambiente integrato locale.

```
# Esecuzione locale di frontend + backend insieme (integrazione/prova).
# Riproduce la topologia di produzione: il browser parla solo col frontend,
# che fa da reverse proxy verso il backend -> niente CORS.
#
# Il DB resta quello remoto: copia .env.example in .env e compila le credenziali.

#   cp .env.example .env && docker compose up --build
# Poi apri http://localhost:8080
services:
  backend:
```

```
build: ./backend
image: bugboard-backend:local
environment:
  SPRING_PROFILES_ACTIVE: docker
  SPRING_DATASOURCE_URL: ${SPRING_DATASOURCE_URL}
  SPRING_DATASOURCE_USERNAME: ${SPRING_DATASOURCE_USERNAME}
  SPRING_DATASOURCE_PASSWORD: ${SPRING_DATASOURCE_PASSWORD}
expose:
  - "8080"          # raggiungibile solo dalla rete interna del compose
# ports:           # scommenta solo per debuggare il backend dall'host
#   - "8080:8080"

frontend:
build: ./frontend
image: bugboard-frontend:local
environment:
  BACKEND_URL: http://backend:8080  # nginx -> service "backend"
ports:
  - "8080:80"          # apri http://localhost:8080
depends_on:
  - backend
```

4 Attività di Testing

Questo documento raccoglie le attività di testing, il primo punto presenta il test plan delle funzionalità significative sotto esame; il secondo punto individua i metodi sottoposti a test di unità `assignIssueToUser`, `aggiungiCommento` e `deleteUser`, mentre l'ultimo punto documenta le strategie adottate per la progettazione dei relativi casi di test. Il sistema oggetto di collaudo è BugBoard, un gestore di issue con backend Spring Boot.

4.1 Strumenti e ambiente di test

I test di unità sono stati realizzati con i seguenti strumenti, già inclusi nella dipendenza `spring-boot-starter-test`:

- **JUnit 5 (Jupiter):** framework di esecuzione dei test e di verifica.
- **Mockito:** libreria di mocking, usata per isolare la classe sotto test sostituendo le sue dipendenze.
- **Test Runner for Java (VS Code):** esecuzione dei test e misura della copertura tramite la funzione `Run with Coverage`.

Non viene utilizzato alcun database reale: i repository e i servizi collaboratori sono simulati con dei mock, così i test risultano rapidi, deterministici e indipendenti dall'ambiente.

4.1.1 Livello e approccio di test

I test realizzati sono unit test ovvero ogni metodo del service viene verificato in isolamento, con le dipendenze sostituite da mock Mockito. Sono state combinate due strategie di progettazione dei casi di test complementari:

- **Testing funzionale (black-box):** i casi sono derivati dalla specifica del metodo tramite il partizionamento dei parametri in classi di equivalenza.
- **Testing strutturale (white-box):** i casi sono completati esaminando il codice, per raggiungere un criterio di copertura strutturale (copertura delle condizioni).

Le due strategie sono complementari: la prima verifica che il programma faccia ciò che deve, la seconda che tutto il codice scritto sia effettivamente esercitato.

4.2 Test Plan

Le funzionalità sotto esame sono tre:

- l'assegnazione di una issue (`assignIssueToUser`).
- l'aggiunta di un commento (`aggiungiCommento`).
- l'eliminazione di un utente (`deleteUser`).

Per ciascuna si forniscono:

- descrizione.
- obiettivi/ambito
- casi di test

l'ambiente di test e i criteri di superamento, comuni a tutte, sono riportati una sola volta.

4.2.1 Ambiente di test (comune)

Elemento	Configurazione
Framework	JUnit 5 (Jupiter)
Mocking	Mockito (MockitoExtension, @Mock, @InjectMocks)
Classi sotto test	bugboard.service.IssueService (assign, commento), bugboard.service.UserService (delete)
Dipendenze mockate	IssueRepository, UserRepository, ISessionService, IssueMapper, BCryptPasswordEncoder, ApplicationEventPublisher
Esecuzione	Test Runner for Java (VS Code) — nessun DB richiesto

4.2.2 Assegnazione di una issue (`assignIssueToUser`)

Descrizione: un admin assegna una issue esistente a uno user, impostando facoltativamente una data di scadenza. L'operazione è soggetta alle seguenti regole:

- Solo un utente autenticato con ruolo ADMIN può eseguire l'assegnazione.
- La issue deve esistere e il destinatario deve esistere ed essere di ruolo USER (non ADMIN né READONLY).
- Una issue in stato CLOSED o DONE non può essere assegnata.

- L'assegnazione (o riassegnazione) di una issue in stato TODO o EXPIRED la riporta in stato IN_PROGRESS, inoltre riprendere una issue scaduta senza nuova scadenza azzera la vecchia scadenza.
- Al termine viene pubblicato un evento di assegnazione (per la notifica al destinatario).

Obiettivi e ambito:

- **In ambito:** controlli di autorizzazione, esistenza di issue e destinatario, verifica del ruolo, verifica dello stato, controllo di versione, transizioni di stato e pubblicazione dell'evento.
- **Fuori ambito:** la persistenza reale su Database, l'invio effettivo della notifica (gestito in modo asincrono da un listener) e l'interfaccia utente.

Casi di test (implementati in IssueServiceTest.java):

ID	Precondizione	Input	Risultato atteso	Postcondizione
A1	Admin autenticato; issue in stato TODO; destinatario USER	issueId valido, email USER, versione corrente	Issue assegnata; stato → IN_PROGRESS; evento pubblicato	Issue persistita con destinatario e stato aggiornati
A2	Sessione non valida	sid nullo/scaduto	ForbiddenException (Sessione non valida)	Nessun accesso al repository; nessuna modifica persistita
A3	Utente autenticato non amministratore	sid di un USER	ForbiddenException (Permessi insufficienti)	Nessun accesso al repository; nessuna modifica persistita
A4	Admin autenticato	issueId inesistente	NotFoundException	Nessuna modifica persistita; nessun evento pubblicato
A5	Admin; issue esistente	email destinatario inesistente	NotFoundException	Issue invariata; nessun salvataggio né evento
A6	Admin; issue esistente	email di un ADMIN o READONLY	BadRequestException (assegnabile solo a USER)	Issue invariata; nessun salvataggio né evento

A7	Admin; issue in stato CLOSED o DONE	dati validi	BadRequestException	Issue invariata nello stato originario; nessun salvataggio né evento
A8	Admin; issue modificata da un altro utente	expectedVersion non più attuale	ConflictException	Issue invariata; l'aggiornamento obsoleto non viene applicato
A9	Admin; issue in stato EXPIRED	dati validi, nessuna nuova scadenza	Scadenza azzerata; stato → IN_PROGRESS	Issue persistita senza scadenza e con nuovo destinatario; evento pubblicato

4.2.3 Aggiunta di un commento (aggiungiCommento)

Descrizione: un utente aggiunge un commento testuale a una issue a lui assegnata. L'operazione è soggetta alle seguenti regole:

- L'utente deve essere autenticato (sessione valida).
- Il testo non può essere nullo né vuoto (anche di soli spazi).
- La issue deve esistere ed è soggetta al controllo di versione (concorrenza ottimistica).
- La issue non può essere in stato DONE, EXPIRED o CLOSED (non commentabile).
- Solo l'utente a cui la issue è assegnata può commentarla (ownership).

Obiettivi e ambito:

- **In ambito:** autenticazione, validazione del testo, esistenza della issue, controllo di versione, verifica dello stato, verifica dell'ownership e aggiunta del commento.
- **Fuori ambito:** persistenza reale su Database e interfaccia utente.

Casi di test (implementati in IssueServiceTest.java):

ID	Precondizione	Input	Risultato atteso	Postcondizione
----	---------------	-------	------------------	----------------

C1	Utente autenticato; issue a lui assegnata in stato commentabile	testo valido, versione corrente	Commento aggiunto	Lista commenti incrementata di un elemento (testo, autore, timestamp); issue persistita
C2	Sessione non valida	sid nullo/scaduto	ForbiddenException	Nessun accesso al repository; lista commenti invariata
C3	Utente autenticato	testo null	BadRequestException	Nessun accesso al repository; lista commenti invariata
C4	Utente autenticato	testo vuoto / soli spazi	BadRequestException	Nessun accesso al repository; lista commenti invariata
C5	Utente autenticato	idIssue inesistente	NotFoundException	Nessuna modifica persistita
C6	Issue modificata da un altro utente	expectedVersion non più attuale	ConflictException	Lista commenti invariata; l'aggiornamento obsoleto non viene applicato
C7	Issue in stato DONE, EXPIRED o CLOSED	dati validi	BadRequestException (non commentabile)	Lista commenti e stato della issue invariati
C8	Issue assegnata ad altro utente o a nessuno	dati validi	ForbiddenException (solo l'assegnatario può commentare)	Lista commenti invariata; nessun salvataggio

4.2.4 Eliminazione di un utente (deleteUser)

Descrizione: un admin elimina un account utente esistente, indicandone l'email. L'operazione è soggetta alle seguenti regole:

- Solo un utente con ruolo ADMIN può eliminare un account.

- L'utente bersaglio deve esistere.
- Non è possibile eliminare un altro amministratore: sono eliminabili solo utenti di ruolo USER o READONLY.

Obiettivi e ambito.

- **In ambito:** controllo di autorizzazione (admin), esistenza del bersaglio, verifica del ruolo eliminabile ed eliminazione.
- **Fuori ambito:** persistenza reale su Database e interfaccia utente.

Casi di test (implementati in UserServiceTest.java):

ID	Precondizione	Input	Risultato atteso	Postcondizione
D1	Admin autenticato; utente USER esistente	email di un USER	Utente eliminato	Utente rimosso dal sistema (delete eseguita sul repository)
D2	Sessione non amministratore	sid di un non-admin	ForbiddenException (Permessi insufficienti)	Nessuna eliminazione: delete mai invocata
D3	Admin autenticato	email inesistente	NotFoundException	Nessuna eliminazione eseguita
D4	Admin; bersaglio di ruolo ADMIN	email di un ADMIN	BadRequestException (un admin non si elimina)	Account ADMIN conservato; delete mai invocata
D5	Admin; bersaglio di ruolo READONLY	email di un READONLY	Utente eliminato	Utente READONLY rimosso dal sistema

4.2.5 Criteri di superamento (comuni)

Un singolo caso è superato se il metodo produce esattamente il risultato atteso (valore restituito / mutazione di stato / eccezione del tipo previsto) e se, al termine dell'esecuzione, lo stato del sistema soddisfa la postcondizione dichiarata (modifiche attese effettivamente applicate oppure, nei casi di errore, assenza di effetti collaterali). Criterio di uscita: tutti i casi di test risultano verdi e la copertura strutturale dei metodi raggiunge la copertura completa delle condizioni.

4.3 Codice dei Test di Unità Automatici

In questa sezione viene riportato il codice sorgente relativo alle classi di test automatiche realizzate per i metodi non banali presi in esame. I metodi dispongono di più parametri in input e i flussi interni vengono validati isolando le classi reali tramite Mockito.

4.3.1 Estratto di IssueServiceTest.java

Di seguito si presenta l'implementazione dei test per i metodi `assignIssueToUser` e `aggiungiCommento`.

Listing 4.1: codice della classe IssueServiceTest

```
*
* Test di unita' per IssueService: le dipendenze sono mockate con Mockito,
* quindi non serve un database.
*
* Per ogni metodo ci sono due gruppi di test. I test "ec_" seguono l'approccio
* black-box delle classi di equivalenza con criterio each-choice: per ogni
* parametro individuiamo le classi, indicate tra parentesi graffe, e ogni
* classe (valida o non valida) compare in almeno un test. I test "cc_" sono
* invece white-box e puntano alla condition coverage, cioè ogni operando
* delle condizioni composte del codice deve risultare sia vero che falso
* almeno una volta. Servono entrambi: nei test black-box in cui combiniamo
* piu' classi non valide si osserva solo la prima eccezione lanciata, perche'
* il primo controllo che fallisce corto-circuita gli altri, quindi il
* black-box da solo non basterebbe a esercitare tutti i controlli.
*/
@ExtendWith(MockitoExtension.class)
class IssueServiceTest {

    @Mock private IssueRepository issueRepository;
    @Mock private UserRepository utenteRepository;
    @Mock private ISessionService sessioneService;
    @Mock private IssueMapper issueMapper;
    @Mock private ApplicationEventPublisher eventPublisher;

    @InjectMocks private IssueService issueService;

    private final UUID adminSid = UUID.randomUUID();
    private final UUID userSid = UUID.randomUUID();

    // ----- helper -----
```

```
private Utente admin(String email) {
    Utente u = new Utente();
    u.setId(1);
    u.setEmail(email);
    u.setRole(Utente.Role.ADMIN);
    return u;
}

private Utente user(int id, String email) {
    Utente u = new Utente();
    u.setId(id);
    u.setEmail(email);
    u.setRole(Utente.Role.USER);
    return u;
}

private Issue issue(StatoIssue stato, long version) {
    Issue i = new Issue();
    i.setId(10);
    i.setStato(stato);
    i.setVersion(version);
    return i;
}

// ----- test di assignIssueToUser -----

/*
 * Parte black-box: classi di equivalenza, criterio each-choice.
 *
 * Per issueId le classi sono {issue esistente}, valida, e {issue
 * inesistente}, non valida perche' porta a NotFound. Per userEmail:
 * {utente esistente con ruolo USER} valida, {utente inesistente} non
 * valida (NotFound), {utente esistente ma di ruolo diverso da USER} non
 * valida (BadRequest, una issue si puo' assegnare solo a un USER). Per
 * expiringDate, che e' opzionale, sono valide entrambe le classi {null}
 * e {data valorizzata}. Per expectedVersion: {null} e {uguale alla
 * versione corrente} valide, {diversa dalla versione corrente} non
 * valida (Conflict). Per adminSID: {sessione di un admin} valida,
 * {sessione nulla} e {sessione di un non admin} non valide (Forbidden).
 */
```

```
* Con l'each-choice bastano tanti test quante sono le classi del
* parametro che ne ha di piu', cioe' tre. Nel primo tutti i parametri
* stanno su una classe valida; negli altri due le classi non valide dei
* vari parametri sono combinate tra loro, quindi si osserva solo la
* prima eccezione: il controllo sulla sessione viene eseguito per primo
* e maschera gli altri. Le classi sulla versione, che qui restano
* mascherate, vengono comunque esercitate dai test white-box cc_assign_5
* e cc_assign_6.
*/

@Test
void ec_assign_1_tuttiValidi() {
    // Tutti i parametri su una classe valida: {issue esistente},
    // {destinatario con ruolo USER}, {scadenza valorizzata},
    // {versione null}, {sessione admin}.
    Issue issue = issue(StatoIssue.TODO, 0L);
    when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("
admin@test.it"));
    when(issueRepository.findById(10)).thenReturn(Optional.of(issue));
    when(utenteRepository.findByEmail("mario@test.it")).thenReturn(Optional.
of(user(2, "mario@test.it")));

    LocalDate scadenza = LocalDate.now().plusDays(7);
    issueService.assignIssueToUser(10, "mario@test.it", scadenza, null,
adminSid);

    assertEquals(scadenza.atTime(23, 59, 59), issue.getDataScadenza());
    assertEquals(StatoIssue.IN_PROGRESS, issue.getStato());
}

@Test
void ec_assign_2_nonValideCombinare() {
    // Classi non valide combinate: {issue inesistente}, {destinatario
    // inesistente} e {sessione nulla}; la versione sta sulla classe valida
    // {uguale alla corrente}. Il controllo sulla sessione fallisce per
    // primo e maschera gli altri, quindi si osserva solo Forbidden.
    when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(null);

    assertThrows(ForbiddenException.class,
        () -> issueService.assignIssueToUser(999, "ghost@test.it", null, 0L,
adminSid));
}
```



```
}

@Test
void ec_assign_3_nonValideCombinare() {
    // Classi non valide combinate: {destinatario di ruolo non USER},
    // {versione diversa dalla corrente} e {sessione di un non admin}.
    // Anche qui la sessione viene controllata per prima e maschera il
    // resto: si osserva solo Forbidden.
    when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(user(5,
"user@test.it"));

    assertThrows(ForbiddenException.class,
        () -> issueService.assignIssueToUser(10, "admin2@test.it", null, 1L,
adminSid));
}

/*
 * Parte white-box: condition coverage.
 *
 * L'obiettivo e' far valutare ogni operando di ogni condizione del
 * metodo, compresi gli helper requireAdmin, findIssueOr404 e
 * checkVersion, sia a vero che a falso. cc_assign_1 e' il caso in cui
 * tutti i controlli passano, quindi copre da solo il lato "si prosegue"
 * di quasi tutte le condizioni; ognuno degli altri test fa scattare un
 * controllo specifico: sessione nulla, utente non admin, issue
 * inesistente, versione diversa e versione uguale, destinatario
 * inesistente o di ruolo sbagliato, i due stati (CLOSED e DONE) in cui
 * la issue non e' assegnabile e infine i due rami della riassegnazione,
 * cioe' issue scaduta che torna in lavorazione e issue gia' in
 * lavorazione che resta dov'e'.
 */

@Test
void cc_assign_1_valida_TODO() { // caso valido: tutti i controlli passano e
da TODO si va in IN_PROGRESS
    Utente dest = user(2, "mario@test.it");
    Issue issue = issue(StatoIssue.TODO, 0L);
    when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("
admin@test.it"));

    when(issueRepository.findById(10)).thenReturn(Optional.of(issue));
    when(utenteRepository.findByEmail("mario@test.it")).thenReturn(Optional.
```

```
of(dest));

    issueService.assignIssueToUser(10, "mario@test.it", null, null, adminSid
);

    assertEquals(StatoIssue.IN_PROGRESS, issue.getStato());
    assertEquals(dest, issue.getAssegnatoA());
    verify(issueRepository).save(issue);
    verify(eventPublisher).publishEvent(any(IssueAssegnataEvent.class));
}

@Test
void cc_assign_2_sessioneNulla_forbidden() { // la sessione non corrisponde
a nessun utente
    when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(null);

    assertThrows(ForbiddenException.class,
        () -> issueService.assignIssueToUser(10, "mario@test.it", null, null
, adminSid));
}

@Test
void cc_assign_3_nonAdmin_forbidden() { // sessione valida ma di un utente
semplice, non admin
    when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(user(5,
"user@test.it"));

    assertThrows(ForbiddenException.class,
        () -> issueService.assignIssueToUser(10, "mario@test.it", null, null
, adminSid));
}

@Test
void cc_assign_4_issueInesistente_notFound() { // la issue cercata non
esiste
    when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("
admin@test.it"));
    when(issueRepository.findById(10)).thenReturn(Optional.empty());

    assertThrows(NotFoundException.class,
        () -> issueService.assignIssueToUser(10, "mario@test.it", null, null
```

```
, adminSid));
}

@Test
void cc_assign_5_versioneDiversa_conflict() { // il client manda una
versione diversa da quella corrente
    when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("
admin@test.it"));
    when(issueRepository.findById(10)).thenReturn(Optional.of(issue(
StatoIssue.TODO, 2L)));

    assertThrows(ConflictException.class,
        () -> issueService.assignIssueToUser(10, "mario@test.it", null, 1L,
adminSid));
}

@Test
void cc_assign_6_versioneUguale_conData() { // versione uguale a quella
corrente e scadenza impostata
    Issue issue = issue(StatoIssue.TODO, 5L);
    when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("
admin@test.it"));
    when(issueRepository.findById(10)).thenReturn(Optional.of(issue));
    when(utenteRepository.findByEmail("mario@test.it")).thenReturn(Optional.
of(user(2, "mario@test.it")));

    LocalDate scadenza = LocalDate.now().plusDays(7);
    issueService.assignIssueToUser(10, "mario@test.it", scadenza, 5L,
adminSid);

    assertEquals(scadenza.atTime(23, 59, 59), issue.getDataScadenza());
    assertEquals(StatoIssue.IN_PROGRESS, issue.getStato());
}

@Test
void cc_assign_7_destinatarioInesistente_notFound() { // l'email indicata
non corrisponde a nessun utente
    when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("
admin@test.it"));
    when(issueRepository.findById(10)).thenReturn(Optional.of(issue(
StatoIssue.TODO, 0L)));
```

```
        when(utenteRepository.findByIdEmail("ghost@test.it")).thenReturn(Optional.  
empty());  
  
        assertThrows(NotFoundException.class,  
            () -> issueService.assignIssueToUser(10, "ghost@test.it", null, null  
, adminSid));  
    }  
  
    @Test  
    void cc_assign_8_destinatarioNonUser_badRequest() { // il destinatario  
        esiste ma e' un admin, non assegnabile  
        when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("  
admin@test.it"));  
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue(  
StatoIssue.TODO, 0L)));  
        when(utenteRepository.findByIdEmail("admin2@test.it")).thenReturn(Optional.  
of(admin("admin2@test.it")));  
  
        assertThrows(BadRequestException.class,  
            () -> issueService.assignIssueToUser(10, "admin2@test.it", null,  
null, adminSid));  
    }  
  
    @Test  
    void cc_assign_9_statoClosed_badRequest() { // issue chiusa, primo operando  
        dell'or sullo stato  
        when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("  
admin@test.it"));  
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue(  
StatoIssue.CLOSED, 0L)));  
        when(utenteRepository.findByIdEmail("mario@test.it")).thenReturn(Optional.  
of(user(2, "mario@test.it")));  
  
        assertThrows(BadRequestException.class,  
            () -> issueService.assignIssueToUser(10, "mario@test.it", null, null  
, adminSid));  
    }  
  
    @Test  
    void cc_assign_10_statoDone_badRequest() { // issue completata, secondo  
        operando dell'or sullo stato
```

```
        when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("
admin@test.it"));
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue(
StatoIssue.DONE, 0L)));
        when(utenteRepository.findByEmail("mario@test.it")).thenReturn(Optional.
of(user(2, "mario@test.it")));

        assertThrows(BadRequestException.class,
            () -> issueService.assignIssueToUser(10, "mario@test.it", null, null
, adminSid));
    }

    @Test
    void cc_assign_11_statoExpired_rispostaInProgress() { // issue scaduta: la
riassegnazione azzerla la scadenza e la rimette in lavorazione
        Issue issue = issue(StatoIssue.EXPIRED, 0L);
        issue.setDataScadenza(LocalDate.now().minusDays(1));
        when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("
admin@test.it"));
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue));
        when(utenteRepository.findByEmail("mario@test.it")).thenReturn(Optional.
of(user(2, "mario@test.it")));

        issueService.assignIssueToUser(10, "mario@test.it", null, null, adminSid
);

        assertNull(issue.getDataScadenza());
        assertEquals(StatoIssue.IN_PROGRESS, issue.getStato());
    }

    @Test
    void cc_assign_12_riassegnaInProgress_restaInProgress() { // issue gia' in
lavorazione e non scaduta
        Issue issue = issue(StatoIssue.IN_PROGRESS, 0L);
        when(sessioneService.getUtenteBySessionId(adminSid)).thenReturn(admin("
admin@test.it"));
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue));
        when(utenteRepository.findByEmail("mario@test.it")).thenReturn(Optional.
of(user(2, "mario@test.it")));

        issueService.assignIssueToUser(10, "mario@test.it", null, null, adminSid
```

```
);

    assertEquals(StatoIssue.IN_PROGRESS, issue.getStato()); // non-TODO e
non-scaduta: lo stato non cambia
    verify(issueRepository).save(issue);
}

// ----- test di aggiungiCommento -----

/*
 * Parte black-box: classi di equivalenza, criterio each-choice.
 *
 * Per idIssue le classi sono {issue esistente}, valida, e {issue
 * inesistente}, non valida (NotFound). Per testo: {testo non vuoto}
 * valida, {null} e {vuoto o di soli spazi} non valide, entrambe
 * BadRequest. Per expectedVersion: {null} e {uguale alla versione
 * corrente} valide, {diversa dalla versione corrente} non valida
 * (Conflict). Per sid: {sessione valida} valida, {sessione non valida o
 * nulla} non valida (Forbidden).
 *
 * Il parametro con piu' classi e' il testo, che ne ha tre, quindi
 * bastano tre test. Le classi non valide sono combinate: in
 * ec_commento_2 fallisce per primo il controllo sulla sessione e si
 * osserva solo Forbidden, in ec_commento_3 il controllo sul testo
 * precede quello sulla versione e si osserva solo BadRequest. I due casi
 * sulla versione, qui mascherati, sono coperti dai white-box
 * cc_commento_6 e cc_commento_7.
 */

@Test
void ec_commento_1_tuttiValidi() {
    // Tutti i parametri su una classe valida: {issue esistente},
    // {testo non vuoto}, {versione null}, {sessione valida}.
    Utente u = user(5, "mario@test.it");
    Issue issue = issue(StatoIssue.IN_PROGRESS, 0L);
    issue.setAssegnatoA(u);
    when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(u);
    when(issueRepository.findById(10)).thenReturn(Optional.of(issue));

    issueService.aggiungiCommento(10, "ok", null, userSid);
}
```

```
        assertEquals(1, issue.getCommento().size());
    }

    @Test
    void ec_commento_2_nonValideCombinare() {
        // Classi non valide combinate: {issue inesistente}, {testo null} e
        // {sessione non valida}; la versione sta su {uguale alla corrente}.
        // Il controllo sulla sessione fallisce per primo, quindi si osserva
        // solo Forbidden.
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(null);

        assertThrows(ForbiddenException.class,
            () -> issueService.aggiungiCommento(999, null, 0L, userSid));
    }

    @Test
    void ec_commento_3_nonValideCombinare() {
        // Classi non valide combinate: {testo vuoto} e {versione diversa
        // dalla corrente}, con sessione e issue valide. Il controllo sul
        // testo precede quello sulla versione, quindi si osserva solo
        // BadRequest.
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(user(5, "
mario@test.it"));

        assertThrows(BadRequestException.class,
            () -> issueService.aggiungiCommento(10, " ", 1L, userSid));
    }

    /*
    * Parte white-box: condition coverage di aggiungiCommento.
    *
    * Come sopra, cc_commento_1 e' il caso valido che copre il lato "si
    * prosegue" di tutti i controlli, mentre gli altri test li fanno fallire
    * uno alla volta: sessione nulla, testo null, testo di soli spazi, issue
    * inesistente, versione diversa e versione uguale, i tre stati in cui
    * non si puo' commentare (DONE, EXPIRED e CLOSED, un operando dell'or
    * ciascuno), i due casi sull'assegnatario (issue senza assegnatario e
    * issue assegnata a qualcun altro) e la lista commenti null che il
    * metodo deve inizializzare.
    */
```

```
@Test
void cc_commento_1_valido() { // caso valido: il commento viene aggiunto con
    testo e autore giusti
        Utente u = user(5, "mario@test.it");
        Issue issue = issue(StatoIssue.IN_PROGRESS, 0L);
        issue.setAssegnatoA(u);
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(u);
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue));

        issueService.aggiungiCommento(10, "Ci sto lavorando", null, userSid);

        assertEquals(1, issue.getCommento().size());
        assertEquals("Ci sto lavorando", issue.getCommento().get(0).getTesto());
        assertEquals("mario@test.it", issue.getCommento().get(0).getAutore());
        verify(issueRepository).save(issue);
    }

@Test
void cc_commento_2_sessioneNulla_forbidden() { // la sessione non
    corrisponde a nessun utente
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(null);

        assertThrows(ForbiddenException.class,
            () -> issueService.aggiungiCommento(10, "ciao", null, userSid));
    }

@Test
void cc_commento_3_testoNull_badRequest() { // testo null
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(user(5, "
mario@test.it"));

        assertThrows(BadRequestException.class,
            () -> issueService.aggiungiCommento(10, null, null, userSid));
    }

@Test
void cc_commento_4_testoVuoto_badRequest() { // testo presente ma di soli
    spazi
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(user(5, "
mario@test.it"));
```



```
        assertThrows(BadRequestException.class,
            () -> issueService.aggiungiCommento(10, " ", null, userSid));
    }

    @Test
    void cc_commento_5_issueInesistente_notFound() { // la issue non esiste
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(user(5, "
mario@test.it"));
        when(issueRepository.findById(10)).thenReturn(Optional.empty());

        assertThrows(NotFoundException.class,
            () -> issueService.aggiungiCommento(10, "ciao", null, userSid));
    }

    @Test
    void cc_commento_6_versioneDiversa_conflict() { // versione diversa da
quella corrente
        Issue issue = issue(StatoIssue.IN_PROGRESS, 2L);
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(user(5, "
mario@test.it"));
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue));

        assertThrows(ConflictException.class,
            () -> issueService.aggiungiCommento(10, "ciao", 1L, userSid));
    }

    @Test
    void cc_commento_7_versioneUguale_ok() { // versione uguale a quella
corrente: il confronto passa
        Utente u = user(5, "mario@test.it");
        Issue issue = issue(StatoIssue.IN_PROGRESS, 5L);
        issue.setAssegnatoA(u);
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(u);
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue));

        issueService.aggiungiCommento(10, "ciao", 5L, userSid);

        assertEquals(1, issue.getCommento().size());
    }

    @Test
```

```
void cc_commento_8_statoDone_badRequest() { // non si commenta una issue
DONE
    Utente u = user(5, "mario@test.it");
    Issue issue = issue(StatoIssue.DONE, 0L);
    issue.setAssegnatoA(u);
    when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(u);
    when(issueRepository.findById(10)).thenReturn(Optional.of(issue));

    assertThrows(BadRequestException.class,
        () -> issueService.aggiungiCommento(10, "ciao", null, userSid));
}

@Test
void cc_commento_9_statoExpired_badRequest() { // non si commenta una issue
EXPIRED
    Utente u = user(5, "mario@test.it");
    Issue issue = issue(StatoIssue.EXPIRED, 0L);
    issue.setAssegnatoA(u);
    when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(u);
    when(issueRepository.findById(10)).thenReturn(Optional.of(issue));

    assertThrows(BadRequestException.class,
        () -> issueService.aggiungiCommento(10, "ciao", null, userSid));
}

@Test
void cc_commento_10_statoClosed_badRequest() { // non si commenta una issue
CLOSED
    Utente u = user(5, "mario@test.it");
    Issue issue = issue(StatoIssue.CLOSED, 0L);
    issue.setAssegnatoA(u);
    when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(u);
    when(issueRepository.findById(10)).thenReturn(Optional.of(issue));

    assertThrows(BadRequestException.class,
        () -> issueService.aggiungiCommento(10, "ciao", null, userSid));
}

@Test
void cc_commento_11_assegnatoANull_forbidden() { // la issue non ha nessun
assegnatario
```

```
        Issue issue = issue(StatoIssue.IN_PROGRESS, 0L);
        issue.setAssegnatoA(null);
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(user(5, "
mario@test.it"));
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue));

        assertThrows(ForbiddenException.class,
            () -> issueService.aggiungiCommento(10, "ciao", null, userSid));
    }

    @Test
    void cc_commento_12_altroAssegnatario_forbidden() { // la issue e' assegnata
        a un altro utente
        Issue issue = issue(StatoIssue.IN_PROGRESS, 0L);
        issue.setAssegnatoA(user(2, "altro@test.it"));
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(user(5, "
mario@test.it"));
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue));

        assertThrows(ForbiddenException.class,
            () -> issueService.aggiungiCommento(10, "ciao", null, userSid));
    }

    @Test
    void cc_commento_13_listaCommentiNull_ok() { // lista commenti null: viene
        inizializzata dal metodo
        Utente u = user(5, "mario@test.it");
        Issue issue = issue(StatoIssue.IN_PROGRESS, 0L);
        issue.setAssegnatoA(u);
        issue.setCommento(null);
        when(sessioneService.getUtenteBySessionId(userSid)).thenReturn(u);
        when(issueRepository.findById(10)).thenReturn(Optional.of(issue));

        issueService.aggiungiCommento(10, "ciao", null, userSid);

        assertEquals(1, issue.getCommento().size());
    }
}
```

4.3.2 Estratto di UserServiceTest.java

Di seguito si presenta l'implementazione dei casi di test per il metodo deleteUser.

Listing 4.2: codice della classe UserServiceTest

```
/*
 * Test di unita' per UserService, in particolare per deleteUser(sid, email).
 * Repository e servizio di sessione sono mockati con Mockito.
 *
 * Come per gli altri servizi ci sono i test black-box sulle classi di
 * equivalenza (criterio each-choice: ogni classe, indicata tra parentesi
 * graffe, compare in almeno un test) e i test white-box per la condition
 * coverage, in cui ogni condizione deve risultare sia vera che falsa.
 * Quando piu' classi non valide finiscono nello stesso test si osserva solo
 * la prima eccezione lanciata, e i controlli cosi' mascherati vengono
 * ripresi dal white-box.
 */
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock private UserRepository utenteRepository;
    @Mock private ISessionService sessioneService;
    @Mock private BCryptPasswordEncoder passwordEncoder;

    @InjectMocks private UserService userService;

    private final UUID sid = UUID.randomUUID();

    private Utente utenteCon(Role ruolo, String email) {
        Utente u = new Utente();
        u.setId(2);
        u.setEmail(email);
        u.setRole(ruolo);
        return u;
    }

    // ----- test di deleteUser -----

    /*
     * Parte black-box: classi di equivalenza, criterio each-choice.
     *
     * Per sid le classi sono {sessione di un admin}, valida, e {sessione di
```

```
* un non admin o nulla}, non valida perche' porta a Forbidden. Per email
* le classi sono tre: {utente esistente con ruolo USER o READONLY},
* valida perche' e' un utente eliminabile, {utente inesistente} non
* valida (NotFound) e {utente esistente con ruolo ADMIN} non valida
* (BadRequest, un amministratore non si puo' eliminare).
*
* Il parametro con piu' classi e' email, che ne ha tre, quindi bastano
* tre test. In ec_delete_2 le classi non valide dei due parametri sono
* combinate: il controllo sull'admin viene eseguito per primo, quindi si
* osserva solo Forbidden e la classe dell'email resta mascherata (viene
* comunque ripresa dai test white-box).
*/

@Test
void ec_delete_1_adminEliminaUser() {
    // Entrambi i parametri su una classe valida: {sessione admin} e
    // {utente eliminabile}, qui di ruolo USER.
    Utente target = utenteCon(Role.USER, "mario@test.it");
    when(sessioneService.isAdmin(sid)).thenReturn(true);
    when(utenteRepository.findByEmail("mario@test.it")).thenReturn(Optional.
of(target));

    userService.deleteUser(sid, "mario@test.it");

    verify(utenteRepository).delete(target);
}

@Test
void ec_delete_2_nonValideCombinare() {
    // Classi non valide combinate: {sessione di un non admin} e {utente
    // inesistente}. Il controllo sull'admin fallisce per primo, quindi
    // si osserva solo Forbidden.
    when(sessioneService.isAdmin(sid)).thenReturn(false);

    assertThrows(ForbiddenException.class,
        () -> userService.deleteUser(sid, "ghost@test.it"));
    verify(utenteRepository, never()).delete(any());
}

@Test
void ec_delete_3_targetAdmin_badRequest() {
```

```
// Classe non valida di email: {utente esistente con ruolo ADMIN},
// un admin non si puo' eliminare.
Utente target = utenteCon(Role.ADMIN, "admin2@test.it");
when(sessioneService.isAdmin(sid)).thenReturn(true);
when(utenteRepository.findByEmail("admin2@test.it")).thenReturn(Optional.
of(target));

assertThrows(BadRequestException.class,
    () -> userService.deleteUser(sid, "admin2@test.it"));
verify(utenteRepository, never()).delete(any());
}

/*
 * Parte white-box: condition coverage di deleteUser. cc_delete_1 e' il
 * caso valido che copre il lato "si prosegue" dei controlli; poi c'e'
 * un test per la sessione non admin, uno per l'email inesistente e due
 * per la condizione composta sul ruolo: con un ADMIN entrambi gli
 * operandi sono veri e scatta il BadRequest, con un READONLY il secondo
 * operando e' falso e l'eliminazione va a buon fine.
 */

@Test
void cc_delete_1_user_ok() { // caso valido: un admin elimina un utente USER
    Utente target = utenteCon(Role.USER, "mario@test.it");
    when(sessioneService.isAdmin(sid)).thenReturn(true);
    when(utenteRepository.findByEmail("mario@test.it")).thenReturn(Optional.
of(target));

    userService.deleteUser(sid, "mario@test.it");

    verify(utenteRepository).delete(target);
}

@Test
void cc_delete_2_nonAdmin_forbidden() { // la sessione non e' di un admin
    when(sessioneService.isAdmin(sid)).thenReturn(false);

    assertThrows(ForbiddenException.class,
        () -> userService.deleteUser(sid, "mario@test.it"));
    verify(utenteRepository, never()).delete(any());
}
```

```
@Test
void cc_delete_3_inesistente_notFound() { // l'email non corrisponde a
nessun utente
    when(sessioneService.isAdmin(sid)).thenReturn(true);
    when(utenteRepository.findByEmail("ghost@test.it")).thenReturn(Optional.
empty());

    assertThrows(NotFoundException.class,
        () -> userService.deleteUser(sid, "ghost@test.it"));
}

@Test
void cc_delete_4_admin_badRequest() { // il bersaglio e' un ADMIN, non
eliminabile
    Utente target = utenteCon(Role.ADMIN, "admin@test.it");
    when(sessioneService.isAdmin(sid)).thenReturn(true);
    when(utenteRepository.findByEmail("admin@test.it")).thenReturn(Optional.
of(target));

    assertThrows(BadRequestException.class,
        () -> userService.deleteUser(sid, "admin@test.it"));
    verify(utenteRepository, never()).delete(any());
}

@Test
void cc_delete_5_readonly_ok() { // il bersaglio e' READONLY, eliminabile
come un USER
    Utente target = utenteCon(Role.READONLY, "ro@test.it");
    when(sessioneService.isAdmin(sid)).thenReturn(true);
    when(utenteRepository.findByEmail("ro@test.it")).thenReturn(Optional.of(
target));

    userService.deleteUser(sid, "ro@test.it");

    verify(utenteRepository).delete(target);
}
}
```

4.4 Strategie di progettazione dei test

4.4.1 Metodi sotto test

Sono stati realizzati test di unità automatici per tre metodi, ciascuno con almeno due parametri:

Metodo	Parametri	N.
assignIssueToUser	issueId, userEmail, expiringDate, expectedVersion, adminSID	5
aggiungiCommento	idIssue, testo, expectedVersion, sid	4
deleteUser	sid, email	2

4.4.2 Testing black-box: classi di equivalenza (criterio each-choice)

Per ciascun parametro del metodo, il dominio di input è stato suddiviso in classi di equivalenza: insiemi di valori che il metodo tratta nello stesso modo. La partizione riguarda esclusivamente i parametri: lo stato interno degli oggetti caricati durante l'esecuzione (per esempio lo stato della issue o la relazione di ownership) non è un parametro e non entra nelle classi black-box è invece coperto dal testing strutturale. Per ogni parametro si distinguono classi valide e classi non valide; nel seguito ogni classe di equivalenza è denotata tra parentesi graffe {}. Come criterio di combinazione è stato adottato l'each-choice: ogni classe di ogni parametro compare in almeno un caso di test, combinando liberamente le classi così da minimizzare il numero di test complessivi.

Motivazione della scelta: Non è stato adottato il criterio all-combinations (prodotto cartesiano di tutte le classi) perché avrebbe generato un numero di casi proibitivo e in larga parte ridondante (per assignIssueToUser si supererebbero le 100 combinazioni), dato che i parametri non interagiscono strettamente tra loro. L'each-choice fornisce la copertura di tutte le classi al costo minimo.

Limite noto (masking): Combinando più classi non valide in uno stesso test, il primo controllo che fallisce interrompe l'esecuzione e maschera gli altri, le classi non valide successive non vengono effettivamente esercitate. Lo stesso vale per le classi {versione uguale} e {versione diversa} di expectedVersion, che nei casi combinati non sono raggiunte. Questo limite dell'each-choice è documentato caso per caso e viene brillantemente compensato dalla copertura strutturale white-box.

assignIssueToUser

Caratteristiche (parametri) e classi di equivalenza:

Parametro	Classi di equivalenza (blocchi)
issueId	{issue esistente}, {issue inesistente}
userEmail (destinatario)	{USER esistente}, {email inesistente}, {utente non-USER}
expiringDate (scadenza, opz.)	{assente (null)}, {valorizzata}
expectedVersion	{null}, {versione uguale alla corrente}, {versione diversa}
adminSID (sessione)	{ADMIN valido}, {sessione nulla}, {utente non-admin}

Casi each-choice (3 test, pari al massimo numero di classi tra i parametri):

Caso	Combinazione di classi	Esito osservato
EC1	issueId={esiste}, userEmail={USER}, expiringDate={valorizzata}, version={null}, adminSID={ADMIN}	Successo: scadenza impostata, issue → IN_PROGRESS
EC2	issueId={inesistente}, userEmail={inesistente}, expiringDate={null}, version={uguale}, adminSID={sessione nulla}	ForbiddenException — resto MASCHERATO
EC3	issueId={esiste}, userEmail={non-USER}, expiringDate={null}, version={diversa}, adminSID={non-admin}	ForbiddenException — resto MASCHERATO

aggiungiCommento

Caratteristiche (parametri) e classi di equivalenza:

Parametro	Classi di equivalenza (blocchi)
idIssue	{issue esistente}, {issue inesistente}
testo	{valido (non vuoto)}, {null}, {vuoto / soli spazi}
expectedVersion	{null}, {versione uguale alla corrente}, {versione diversa}
sid (sessione)	{valido}, {non valido (incl. null)}

Casi each-choice (3 test):

Caso	Combinazione di classi	Esito osservato
EC1	idIssue={esiste}, testo={valido}, version={null}, sid={valido}	Successo: commento aggiunto
EC2	idIssue={inesistente}, testo={null}, version={uguale}, sid={nullo}	ForbiddenException — resto MASCHERATO
EC3	idIssue={esiste}, testo={vuoto}, version={diversa}, sid={valido}	BadRequestException — resto MASCHERATO

deleteUser

Caratteristiche (parametri) e classi di equivalenza.

Parametro	Classi di equivalenza (blocchi)
sid (sessione)	{amministratore}, {non amministratore}
email (utente da eliminare)	{esistente eliminabile (ruolo USER o READONLY)}, {inesistente}, {esistente con ruolo ADMIN}

Casi each-choice (3 test):

Caso	Combinazione di classi	Esito osservato
EC1	sid={admin}, email={eliminabile (USER)}	Successo: utente eliminato
EC2	sid={non-admin}, email={inesistente}	ForbiddenException — resto MASCHERATO
EC3	sid={admin}, email={esistente con ruolo ADMIN}	BadRequestException (un admin non si elimina)

4.4.3 Testing white-box: criteri di copertura strutturale

I criteri di copertura strutturale, dal più debole al più forte, sono:

- copertura delle istruzioni (statement).
- copertura delle decisioni/rami (branch).
- copertura delle condizioni (condition).

I primi sono sottoinsiemi dei successivi.

Criterio adottato: copertura completa delle condizioni (condition coverage). Ogni condizione viene valutata sia a vero sia a falso almeno una volta. Questo criterio implica anche la copertura delle istruzioni e dei rami sui metodi testati. Le tabelle seguenti tracciano, per ogni condizione, il test che lo rende vero e quello che lo rende falso (i nomi fanno riferimento ai metodi di test).

assignIssueToUser

Condizione / operando	Vero	Falso
requireUser: user == null	cc_assign_2	cc_assign_1
requireAdmin: role != ADMIN	cc_assign_3	cc_assign_1
findIssueOr404: Optional vuoto	cc_assign_4	cc_assign_1
checkVersion: expectedVersion != null	cc_assign_5	cc_assign_1
checkVersion: !expectedVersion.equals(v)	cc_assign_5	cc_assign_6
findByEmail destinatario: Optional vuoto	cc_assign_7	cc_assign_1
destinatario: role != USER	cc_assign_8	cc_assign_1
stato == CLOSED	cc_assign_9	cc_assign_1
stato == DONE	cc_assign_10	cc_assign_1
expiringDate != null	cc_assign_6	cc_assign_1
riattivazione: stato == TODO	cc_assign_1	cc_assign_11
eraScaduta (else-if e riattivazione)	cc_assign_11	cc_assign_12

aggiungiCommento

Condizione / operando	Vero	Falso
requireUser: user == null	cc_commento_2	cc_commento_1
testo == null	cc_commento_3	cc_commento_1
testo.trim().isEmpty()	cc_commento_4	cc_commento_1
findIssueOr404: Optional vuoto	cc_commento_5	cc_commento_1
checkVersion: expectedVersion != null	cc_commento_6	cc_commento_1
checkVersion: !expectedVersion.equals(v)	cc_commento_6	cc_commento_7
stato == DONE	cc_commento_8	cc_commento_1
stato == EXPIRED	cc_commento_9	cc_commento_1
stato == CLOSED	cc_commento_10	cc_commento_1
ownership: assegnatoA == null	cc_commento_11	cc_commento_1
ownership: !assegnatoA.getId().equals(id)	cc_commento_12	cc_commento_1
lista commenti == null	cc_commento_13	cc_commento_1

deleteUser

Condizione / operando	Vero	Falso
requireAdmin: !isAdmin(sid)	cc_delete_2	cc_delete_1
findByEmail: Optional vuoto	cc_delete_3	cc_delete_1
ruolo != USER	cc_delete_4	cc_delete_1

ruolo != READONLY	cc_delete_4	cc_delete_5
-------------------	-------------	-------------